

The Free Web-Making Mega Handbook

183 researched resource entries + 430 original code recipes

Research snapshot: 2026-08-07

A step-by-step, license-aware field guide to HTML, CSS, JavaScript, responsive UI, animation, component sources, templates, playgrounds, generators, icons, and screenshot-inspired builds.

How to use this handbook

The directory is a researched snapshot, not a claim that the changing Internet can be literally exhausted. It intentionally distinguishes open source, free tools, freemium resources, playgrounds with author-controlled licenses, and social/comment-gated code leads.

The 430 numbered recipes are original instructional examples written for this handbook. External pages are research/reference links; the code recipes below are not scraped copies.

For each recipe: run it, change one value, break it deliberately, restore it, then test keyboard, narrow viewport, zoom, reduced motion, and failure states when relevant. That turns a snippet into a skill.

License survival rules

- Free to view is not automatically free to republish.
- A public GitHub repository needs a LICENSE file before you assume reuse rights.
- A comment/DM/Telegram download does not itself grant a license.
- Freemium sites can have free components beside paid or commercially restricted material.
- Record source URL, version/commit, license, required notices, and dependencies before shipping third-party code.

Study roadmap

- HTML foundations: snippets 001-072
- CSS micro-recipes: snippets 073-222
- JavaScript browser recipes: snippets 223-380
- UI mini-projects: snippets 381-430

Part I - Researched free web-code/resource directory

183 entries. URLs are clickable. Status is deliberately cautious when the current item/license can vary.

Screenshot-derived

001. aduok_code (Instagram) - Creator lead

https://www.instagram.com/aduok_code/

Screenshot 1: responsive 3D photo gallery; creator says source is linked from bio.

002. aduok_code Telegram - Free-code channel lead

https://t.me/aduok_code

Search results tied this Telegram channel to the creator's source-code posts. Check each post/license.

003. FreeFrontend - aduok responsive fluid gallery - Free code listing

<https://freefrontend.com/code/responsive-fluid-photo-gallery-grid-2026-03-30/>

A gallery entry attributed to aduok; use the license shown on the individual entry.

004. CodeWriterDev (Instagram) - Creator lead

<https://www.instagram.com/codewriterdev/>

Screenshot 2: CSS cheat sheet. Posts point to a bio website for some source code; no standalone code domain was reliably resolved.

005. Coding Stella - Tesla-style hero tutorial - Direct tutorial + code

<https://codingstella.com/how-to-create-animated-tesla-landing-page-using-html-css-and-js/>

Screenshot 3 concept. Tutorial exposes HTML/CSS/JS and project assets; review site terms before redistribution.

006. kontsybko_web (Instagram) - Creator lead

https://www.instagram.com/kontsybko_web/

Screenshot 4: Neon Launch Gallery. Source is offered through comments/bio/Telegram rather than a verified open repository.

007. Code with Nazia (Instagram) - Creator lead

<https://www.instagram.com/codewithnazia/>

Screenshot 5. Creator directs viewers to YouTube/website/Telegram for free source; exact external domain was not reliably resolved in search.

008. CodeXR (Instagram) - Creator lead

<https://www.instagram.com/code.xr/>

Screenshot 6: Login Animation V2. Comment-to-code flow; no verified open repository found.

009. Code Devotee (Instagram reel) - Creator lead

<https://www.instagram.com/reel/DAYGFuvvltc/>

Screenshot 7 is consistent with Code Devotee material; search results point to source via Telegram in bio. Check license per item.

010. Coding Stella - animated 404 tutorial - Direct tutorial + code

<https://codingstella.com/how-to-create-animated-404-page-not-found-using-html-and-css/>

Screenshot 8 concept, with HTML/CSS tutorial code.

011. codewith_muhilan (Instagram) - Creator lead

https://www.instagram.com/codewith_muhilan/

Screenshot 9: Modern Carousel; creator posts source-code offers and maintains public code/resources elsewhere.

012. codewithmuhilan GitHub - Public repositories

<https://github.com/codewithmuhilan>

Public repositories include source associated with @codewith_muhilan. Check each repository license.

013. Lightswind - Free + open-source core

<https://lightswind.com/>

Codewithmuhilan-associated copy/paste UI collection; free components are separated from paid offerings.

014. kevin.snippet (Instagram) - Creator lead

<https://www.instagram.com/kevin.snippet/>

Screenshot 10: reel describing three CSS websites.

015. Animista - Free generated CSS

<https://animista.net/>

Hidden site #1 from screenshot 10. Build CSS animations visually and copy/download generated code; site states generated code is free for personal/commercial use under its listed license.

016. CSS Grid Generator - Free generator

<https://cssgrid-generator.netlify.app/>

Hidden site #2 from screenshot 10. Visual CSS Grid builder.

017. CSS Gradient - Free generator

<https://cssgradient.io/>

Hidden site #3 from screenshot 10. Visual gradient builder with copyable CSS.

Learn + reference

018. MDN Web Docs - Free reference

<https://developer.mozilla.org/en-US/>

HTML, CSS, JavaScript, accessibility, APIs, examples, and compatibility tables. MDN content/code licensing applies.

019. MDN Learn Web Development - Free curriculum

https://developer.mozilla.org/en-US/docs/Learn_web_development

Structured front-end learning path with examples and exercises.

020. MDN content repository - Open-source documentation repo

<https://github.com/mdn/content>

Source repository for MDN content; review the repository's content/code licenses.

021. web.dev Learn - Free official courses

<https://web.dev/learn>

Google's structured courses for HTML, CSS, JavaScript, accessibility, performance, PWA and more.

022. web.dev Learn CSS - Free official course

<https://web.dev/learn/css>

Evergreen CSS course and reference.

023. 1-Line Layouts - Free demos

<https://1linelayouts.com/>

Ten compact modern CSS layout and sizing techniques with editable demos.

024. W3C CSS - Open-web standard reference

<https://www.w3.org/Style/CSS/>

CSS specifications, working-group material, and official links.

025. WHATWG HTML Living Standard - Open standard

<https://html.spec.whatwg.org/>

Authoritative living HTML standard.

026. WAI Tutorials - Free accessibility guidance

<https://www.w3.org/WAI/tutorials/>

Practical accessibility patterns from W3C WAI.

027. ARIA Authoring Practices Guide - Free accessibility patterns

<https://www.w3.org/WAI/ARIA/apg/>

Accessible widget patterns, keyboard behavior, and examples.

028. Can I Use - Free compatibility reference

<https://caniuse.com/>

Browser support tables for web platform features.

029. JavaScript.info - Free learning site

<https://javascript.info/>

Deep JavaScript tutorials; material has its own content license, so do not assume unrestricted republication.

030. Patterns.dev - Free pattern reference

<https://www.patterns.dev/>

Modern web design, rendering, and JavaScript pattern explanations.

031. W3Schools - Free learning + examples

<https://www.w3schools.com/>

Tutorials and Tryit examples; W3Schools content copyright/terms apply.

032. W3Schools Tryit - Free playground

<https://www.w3schools.com/tryit/>

Browser-based HTML/CSS/JS examples and exercises.

033. CSS-Tricks - Free articles + demos

<https://css-tricks.com/>

Front-end tutorials and demos; individual article/code licensing applies.

034. A11Y Project - Free accessibility resource

<https://www.a11yproject.com/>

Community-driven accessibility checklist and articles.

035. webhint - Open-source guidance/tooling

<https://webhint.io/>

Checks accessibility, compatibility, performance and web best practices.

Code galleries + playgrounds

036. FreeFrontend - Curated free code

<https://freefrontend.com/>

Large catalog of HTML/CSS/JS examples. Check the license shown on every individual entry.

037. CodeMyUI - Curated snippets

<https://codemyui.com/>

Front-end UI snippets with HTML/CSS and often JavaScript/library dependencies; attribution/license varies by item.

038. CodeHim - Open-source code catalog

<https://codehim.com/>

Web design scripts/components; site describes published codes as open source. Still review the item license.

039. FreeFrontend.dev - Free components

<https://freefrontend.dev/code/>

Free components/templates distinct from FreeFrontend.com.

040. 77 Coding Projects - Free examples

<https://77coding.github.io/projects.html>

Ready-to-run front-end project examples; verify repository/item license before redistribution.

041. CodePen - Free public playground

<https://codepen.io/>

Public Pens are automatically MIT-licensed under CodePen's licensing policy; private Pens have no implicit license.

042. CodePen licensing - License reference

<https://blog.codepen.io/documentation/licensing/>

Primary reference for CodePen Pen licensing.

043. JSFiddle - Free playground; license varies

<https://jsfiddle.net/>

Excellent for running examples, but JSFiddle's terms do not make every user's code open source. Check author/license.

044. JS Bin - Free playground; license varies

<https://jsbin.com/>

Live HTML/CSS/JS sandbox. Treat user-authored bin reuse as license-dependent.

045. PlayCode - Free playground; license varies

<https://playcode.io/>

Browser IDE for HTML/CSS/JS and frameworks. User project licensing varies.

046. CodeSandbox - Free tier; license varies

<https://codesandbox.io/>

Cloud development sandboxes. A public sandbox is not automatically reusable unless its license says so.

047. StackBlitz - Free tier; license varies

<https://stackblitz.com/>

Browser-based development environments. Check project/repository license.

048. GitHub website-template topic - Repository index

<https://github.com/topics/website-template>

Huge source-code index. Always check the LICENSE file in each repository.

049. GitHub html-css-javascript topic - Repository index

<https://github.com/topics/html-css-javascript>

Discover public front-end projects; public does not automatically mean open-source.

050. GitHub css-animation topic - Repository index

<https://github.com/topics/css-animation>

Discover CSS animation projects; license per repository.

CSS + SVG generators

051. CSSMatic - Free CSS generators

<https://www.cssmatic.com/>

Gradient, radius, pattern, noise, glass and related copy-ready CSS tools.

052. Layoutit Grid - Free open-source generator

<https://grid.layoutit.com/>

Interactive CSS Grid designer that exports HTML/CSS; project repository is MIT licensed.

053. CSS Grid Generator - Free generator

<https://cssgridgenerator.io/>

Drag-and-drop grid tool with Copy HTML and Copy CSS actions.

054. Clippy - Free generator

<https://bennettfeely.com/clippy/>

Visual clip-path shape editor that emits CSS.

055. Grabient - Free gradient tool
<https://grabient.com/>
Build gradients and copy CSS or export graphics.
056. Gradient Magic - Free CSS gallery
<https://www.gradientmagic.com/>
Free gallery of complex CSS gradients.
057. WebGradients - Free gradient collection
<https://webgradients.com/>
180 gradients with copyable CSS; site states personal/commercial use is free.
058. MagicPattern Gradient Generator - Free generator
<https://www.magicpattern.design/tools/gradient-generator>
Free no-signup linear/radial/conic gradient builder with CSS export.
059. Hero Patterns - Free CC BY 4.0 patterns
<https://heropatterns.com/>
Repeatable SVG backgrounds; site states it will remain free and lists CC BY 4.0.
060. SVG Backgrounds - Free + paid graphics
<https://www.svgbackgrounds.com/>
Copy/paste customizable SVG backgrounds; review the specific asset/license tier.
061. Haikei - Free SVG generator
<https://haikei.app/>
No-signup SVG background generators with SVG/PNG output.
062. Get Waves - Free SVG generator
<https://www.getwaves.io/>
Generate customizable SVG waves.
063. BGJar - Free SVG generator
<https://bgjar.com/>
Generate SVG backgrounds for sites and apps.
064. Blobmaker - Free SVG generator
<https://www.blobmaker.app/>
Generate organic SVG blobs and copy/download output.
065. Neumorphism.io - Free CSS generator
<https://neumorphism.io/>
Visual neumorphic box-shadow generator with copyable CSS.
066. Glassmorphism Generator - Free CSS generator
<https://hype4.academy/tools/glassmorphism-generator>
Visual glass/frosted effect generator.
067. Fancy Border Radius - Free CSS generator
<https://9elements.github.io/fancy-border-radius/>
Build organic shapes using eight-value border-radius syntax.
068. Smooth Shadows - Free CSS generator
<https://smoothshadows.com/>
Tune layered shadows and copy the CSS.
069. CSS Portal - Free CSS generators
<https://www.cssportal.com/css-generators.php>
Collection of visual CSS generators; generated declarations are copy-ready.
070. Keyframes.app - Free CSS tools
<https://keyframes.app/>
Animation, color/shadow and other CSS authoring helpers.
071. cubic-bezier.com - Free easing tool
<https://cubic-bezier.com/>
Preview and copy cubic-bezier timing functions.
072. CSS Loaders - Free CSS source
<https://cssloaders.github.io/>
Collection of loaders/spinners with inspectable CSS source.
073. SpinKit - Free CSS loader source
<https://tobiasahlin.com/spinkit/>
CSS loading indicators with source examples.

CSS frameworks

074. Bootstrap - Open source

<https://getbootstrap.com/>

Popular responsive CSS/JS framework.

075. Tailwind CSS - Open source core

<https://tailwindcss.com/>

Utility-first CSS framework; some ecosystem templates are paid separately.

076. Bulma - Open source

<https://bulma.io/>

CSS-only responsive framework.

077. Foundation - Open source

<https://get.foundation/>

Responsive front-end framework.

078. UIKit - Open source

<https://getuikit.com/>

Modular front-end framework.

079. Pure.css - Open source

<https://pure-css.github.io/>

Small CSS modules for responsive sites.

080. Pico CSS - Open source

<https://picocss.com/>

Minimal semantic CSS framework.

081. Materialize - Open source

<https://materializecss.com/>

Material-design-inspired front-end framework.

082. Primer CSS - Open source

<https://primer.style/>

GitHub's CSS design-system foundation.

083. Open Props - Open source

<https://open-props.style/>

CSS custom-property design tokens and utilities.

084. Shoelace - Open source web components

<https://shoelace.style/>

Framework-agnostic accessible web components; project continues as Web Awesome ecosystem, so check current project status.

Copy-paste UI

085. daisyUI - Open-source core

<https://daisyui.com/>

Component classes for Tailwind CSS; themes and ecosystem extras vary.

086. Uiverse - Free community UI

<https://uiverse.io/>

Copyable HTML/CSS/Tailwind/React UI elements; review each community item/license.

087. Flowbite - Open-source core + Pro

<https://flowbite.com/docs/getting-started/introduction/>

Tailwind components with JavaScript; paid blocks exist separately.

088. Preline UI - Free open-source core + Pro

<https://preline.co/>

Large Tailwind component set with paid blocks/templates separated.

089. Mamba UI - Free components

<https://mambaui.com/>

Copy/paste Tailwind components and templates.

090. HyperUI - Free components

<https://www.hyperui.dev/>

Open Tailwind component collection.

091. Tailwind Toolbox - Free resources + templates

<https://www.tailwindtoolbox.com/>

Tailwind components/templates; license varies by resource.

092. Float UI - Free components/templates

<https://floatui.com/>

Tailwind/React UI components and templates; confirm current item license.

093. Meraki UI - Free Tailwind components

<https://merakiui.com/>

Copy-ready responsive Tailwind components.

094. Tailblocks - Free Tailwind blocks

<https://tailblocks.cc/>

Copy-ready Tailwind blocks/components.

095. Tailwind Components - Community snippets; license varies

<https://tailwindcomponents.com/>

Community Tailwind snippets. Check creator/license per component.

096. Tailwind Awesome - Free + paid resources

<https://www.tailwindawesome.com/>

Tailwind templates/components with mixed licensing/price; filter and verify each item.

097. FlyonUI - Open-source component library

<https://flyonui.com/>

Semantic Tailwind component library with JS plugins.

098. TW Elements - Free/open-source core

<https://tw-elements.com/>

Bootstrap-style components built for Tailwind; premium products may be separate.

099. Cruip free templates - Free + paid

<https://cruip.com/free-templates/>

Free Tailwind/Next/React landing templates plus paid products; verify individual license.

100. shadcn/ui - Open code / copy into project

<https://ui.shadcn.com/>

Accessible React components distributed as source rather than a locked dependency.

101. Magic UI - Free open-source core + Pro

<https://magicui.design/>

150+ animated React/TypeScript/Tailwind/Motion components; Pro blocks exist separately.

102. Aceternity UI - 200+ free components + paid

<https://ui.aceternity.com/components>

Copy-paste animated React/Next/Tailwind/Motion components; premium packs exist separately.

103. 21st.dev - Registry; mixed licenses/pricing

<https://21st.dev/>

Community component registry. Check the source package/license for every component.

104. Hover.dev - Free + Pro

<https://www.hover.dev/>

React/Tailwind/Motion components; distinguish free components from Pro content.

105. Kokonut UI - Free/open-source components

<https://kokonutui.com/>

Copy-paste React components and UI patterns.

106. React Bits - Free open source

<https://reactbits.dev/>

Animated, interactive React components/effects with source.

107. Motion Primitives - Open source

<https://motion-primitives.com/>

Animated React/Next/Tailwind primitives.

108. Animate UI - Open source

<https://animate-ui.com/>

Animated React components distributed as source.

109. CuiCui - Open source / MIT

<https://github.com/damien-schneider/cuicui>

Copy/paste React component library.

Component libraries

110. Radix UI - Open source

<https://www.radix-ui.com/>

Accessible unstyled React primitives.

111. Base UI - Open source

<https://base-ui.com/>

Accessible unstyled React components.

112. Ark UI - Open source
<https://ark-ui.com/>
Headless components for multiple frameworks.
113. Headless UI - Open source
<https://headlessui.com/>
Unstyled accessible React/Vue components from Tailwind Labs.
114. React Aria - Open source
<https://react-spectrum.adobe.com/react-aria/>
Accessible React primitives/hooks/components from Adobe.
115. MUI - Open-source core + paid
<https://mui.com/>
React Material UI component system; premium products are separate.
116. Chakra UI - Open source
<https://chakra-ui.com/>
Accessible React component system.
117. Mantine - Open source
<https://mantine.dev/>
React component and hooks library.
118. HeroUI - Open source core
<https://www.heroui.com/>
React UI library formerly known as NextUI.
119. Vuetify - Open source
<https://vuetifyjs.com/>
Vue component framework.
120. Quasar - Open source
<https://quasar.dev/>
Vue-based cross-platform framework and components.
121. PrimeVue - Open-source core + paid
<https://primevue.org/>
Vue component library with optional premium templates/blocks.
122. Element Plus - Open source
<https://element-plus.org/>
Vue 3 component library.
123. Naive UI - Open source
<https://www.naiveui.com/>
Vue 3 component library.
124. shadcn-vue - Open code
<https://www.shadcn-vue.com/>
Vue port of copy-into-project component patterns.
125. shadcn-svelte - Open code
<https://www.shadcn-svelte.com/>
Svelte copy-into-project component collection.
126. Bits UI - Open source
<https://bits-ui.com/>
Headless accessible Svelte component primitives.
127. Skeleton - Open source core
<https://www.skeleton.dev/>
Svelte/Tailwind design system and components.

Templates

128. HTML5 UP - 100% free / Creative Commons
<https://html5up.net/>
Responsive HTML5/CSS3 templates; follow the site's Creative Commons attribution terms.
129. Start Bootstrap - Free open-source templates + paid
<https://startbootstrap.com/>
Free Bootstrap themes/templates/snippets plus paid products.
130. ThemeWagon free templates - Free templates
<https://themewagon.com/theme-price/free/>
Large free HTML/Bootstrap/Tailwind catalog; site advertises free commercial use, but still check the individual template license.

131. Tooplate - Free HTML templates

<https://www.tooplate.com/>

Free instant HTML template downloads without login; review site/template terms.

132. TemplateMo - Free HTML templates

<https://templatemo.com/>

Free HTML/CSS/Bootstrap templates; review terms and attribution requirements.

133. Colorlib templates - Free + premium

<https://colorlib.com/wp/templates/>

Large free template catalog mixed with commercial products; verify each template license.

134. Uldeck - Free + premium

<https://uideck.com/>

HTML/Tailwind/Bootstrap templates and UI kits; filter free offerings and verify item license.

135. W3Layouts - Free + paid

<https://w3layouts.com/>

Large template catalog with licensing/attribution tiers; read the selected template's terms.

136. AdminLTE - Open source

<https://adminlte.io/>

Bootstrap admin dashboard template.

137. Tabler - Open source

<https://tabler.io/admin-template>

Responsive Bootstrap admin/dashboard UI kit.

138. CoreUI - Open-source core + Pro

<https://coreui.io/>

Dashboard/components with free open-source and commercial tiers.

139. BootstrapMade - Free + paid

<https://bootstrapmade.com/>

Bootstrap templates; free licenses can require attribution, so check terms before use.

140. Themefisher - Free + premium

<https://themefisher.com/free-bootstrap-templates>

Free Bootstrap template collection; license/attribution varies by template.

Animation + interaction

141. GSAP - Free under GSAP standard license

<https://gsap.com/>

GSAP and former premium plugins are free to use under current standard license; it is not an OSI open-source license.

142. Motion - Open source / MIT

<https://motion.dev/>

Animation library for JavaScript, React and Vue.

143. Anime.js - Open source / MIT

<https://animejs.com/>

Lightweight JavaScript animation engine.

144. Three.js - Open source

<https://threejs.org/>

WebGL/3D rendering library with extensive examples.

145. p5.js - Free open source

<https://p5js.org/>

Creative-coding JavaScript library.

146. Swiper - Open source / MIT

<https://swiperjs.com/>

Touch slider/carousel library.

147. Splide - Open source / MIT

<https://splidejs.com/>

Accessible slider/carousel library.

148. Embla Carousel - Open source / MIT

<https://www.embla-carousel.com/>

Lightweight carousel engine.

149. Keen Slider - Open source

<https://keen-slider.io/>

Touch slider with framework integrations.

150. Glide.js - Open source
<https://glidejs.com/>
Dependency-free ES6 slider/carousel.
151. AutoAnimate - Open source
<https://auto-animate.formkit.com/>
Zero-config animations for DOM additions, removals and movement.
152. AOS - Open source
<https://michalsnik.github.io/aos/>
Animate-on-scroll library.
153. ScrollReveal - Free library; check license
<https://scrollrevealjs.org/>
Scroll-triggered reveal animations.
154. SortableJS - Open source
<https://sortablejs.github.io/Sortable/>
Drag-and-drop list reordering.
155. Floating UI - Open source
<https://floating-ui.com/>
Positioning engine for tooltips, popovers, dropdowns and floating elements.
156. Tippy.js - Open source / legacy
<https://atomiks.github.io/tippyjs/>
Tooltip/popover library; site notes it is no longer actively developed, so prefer maintained alternatives for new work.
157. Lottie Web - Open source
<https://github.com/airbnb/lottie-web>
Render After Effects/Lottie vector animations on the web.
158. tsParticles - Open source
<https://particles.js.org/>
Particle effects engine with web demos.
159. VanillaTilt - Open source
<https://micku7zu.github.io/vanilla-tilt.js/>
Dependency-free 3D tilt interaction.
160. Typed.js - Open source
<https://mattboldt.com/demos/typed-js/>
Typing animation library.
161. Barba.js - Open source
<https://barba.js.org/>
Page transition library for smooth navigation experiences.
162. Lenis - Open source
<https://lenis.darkroom.engineering/>
Smooth scrolling library.
163. Animate.css - Free with Hippocratic License
<https://animate.style/>
Ready-to-use CSS animation classes. Current site lists Hippocratic License 2.1; do not label it MIT.
164. Hover.css - Personal/open-source free; commercial license sold
<https://ianlunn.github.io/Hover/>
CSS hover effects. Current license terms distinguish free personal/open-source use from commercial use.

Icons + fonts

165. Heroicons - Open source
<https://heroicons.com/>
SVG icons by Tailwind Labs.
166. Lucide - Open source
<https://lucide.dev/>
Community SVG icon toolkit.
167. Tabler Icons - Open source / MIT
<https://tabler.io/icons>
Thousands of SVG icons for personal/commercial use under MIT.
168. Phosphor Icons - Open source / MIT
<https://phosphoricons.com/>
Flexible SVG icon family.

169. Iconoir - Free open source
<https://iconoir.com/>
Large SVG icon library.
170. Remix Icon - Free open source
<https://remixicon.com/>
Neutral SVG icon system; site states icons are free to use.
171. Bootstrap Icons - Open source
<https://icons.getbootstrap.com/>
Official Bootstrap SVG icon library.
172. Simple Icons - Open-source project; brand rights still apply
<https://simpleicons.org/>
SVG brand icons. Project license does not override trademark/brand rules.
173. Material Symbols - Open-source icon set
<https://fonts.google.com/icons>
Google Material symbols/icons; consult repository license for files.
174. Iconify - Open-source aggregator
<https://icon-sets.iconify.design/>
Hundreds of thousands of icons from open-source sets; license is shown per icon set.
175. Iconbuddy - Open-source icon aggregator
<https://iconbuddy.com/>
Search/customize/export SVG and framework formats; check the source set's license.
176. css.gg - Open source
<https://css.gg/>
CSS/SVG icon library.
177. Feather Icons - Open source
<https://feathericons.com/>
Minimal SVG icon set; largely superseded by the active Lucide fork but still useful.
178. Font Awesome Free - Free/open licenses for free set
<https://fontawesome.com/search?o=r&m=free>
Free icon set; icon/font/code portions use different licenses, so follow Font Awesome licensing.
179. Boxicons - Open-source core
<https://boxicons.com/>
Web-friendly icon library.
180. Ionicons - Open source
<https://ionic.io/ionicons>
SVG icon pack from Ionic.
181. Octicons - Open source
<https://primer.style/octicons/>
GitHub's icon set.
182. Google Fonts - Open-source fonts
<https://developers.google.com/fonts>
Google states Fonts catalog typefaces are open source and usable commercially/noncommercially; check each font license.
183. Fontsource - Open-source font packages
<https://fontsource.org/>
Self-hostable packages for open-source fonts.

Part II - 430 original code recipes

Each recipe starts on a fresh page for print-friendly study and annotation. The final 50 are broader mini-project scaffolds; the first ten deliberately mirror the ideas in your screenshots without copying creator source.

HTML foundations

Recipes 001-072

Snippet 001 - Minimal HTML5 document

Goal: Use minimal html5 document as a small, standards-first building block.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title</title>
</head>
<body>
  <main>Hello, web.</main>
</body>
</html>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 002 - Semantic page landmarks

Goal: Use semantic page landmarks as a small, standards-first building block.

```
<header>Site header</header>
<nav aria-label="Primary">...</nav>
<main id="main">Main content</main>
<footer>Site footer</footer>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 003 - Skip link

Goal: Use skip link as a small, standards-first building block.

```
<a class="skip-link" href="#main">Skip to main content</a>  
<main id="main" tabindex="-1">...</main>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 004 - Heading hierarchy

Goal: Use heading hierarchy as a small, standards-first building block.

```
<h1>Account settings</h1>
<section>
  <h2>Profile</h2>
  <h3>Public name</h3>
</section>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 005 - Descriptive link

Goal: Use descriptive link as a small, standards-first building block.

```
<a href="/pricing">Compare plans and pricing</a>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 006 - External link with safe target

Goal: Use external link with safe target as a small, standards-first building block.

```
<a href="https://example.com" target="_blank" rel="noopener">Open reference</a>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 007 - Responsive image with alt text

Goal: Use responsive image with alt text as a small, standards-first building block.

```

```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 008 - Decorative image

Goal: Use decorative image as a small, standards-first building block.

```

```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 009 - Picture art direction

Goal: Use picture art direction as a small, standards-first building block.

```
<picture>
  <source media="(min-width: 60rem)" srcset="hero-wide.webp">
  
</picture>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 010 - Responsive srcset

Goal: Use responsive srcset as a small, standards-first building block.

```

```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 011 - Figure with caption

Goal: Use figure with caption as a small, standards-first building block.

```
<figure>
  
  <figcaption>Orders by month, January-June.</figcaption>
</figure>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 012 - Unordered list

Goal: Use unordered list as a small, standards-first building block.

```
<ul>
  <li>Fast setup</li>
  <li>Accessible defaults</li>
  <li>No framework required</li>
</ul>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 013 - Ordered procedure

Goal: Use ordered procedure as a small, standards-first building block.

```
<ol>
  <li>Download the files.</li>
  <li>Open <code>index.html</code>.</li>
  <li>Test at narrow and wide widths.</li>
</ol>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 014 - Description list

Goal: Use description list as a small, standards-first building block.

```
<dl>
  <dt>HTML</dt><dd>Structure and meaning.</dd>
  <dt>CSS</dt><dd>Presentation and layout.</dd>
</dl>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 015 - Accessible data table

Goal: Use accessible data table as a small, standards-first building block.

```
<table>
  <caption>Plan limits</caption>
  <thead><tr><th scope="col">Plan</th><th scope="col">Projects</th></tr></thead>
  <tbody><tr><th scope="row">Free</th><td>3</td></tr></tbody>
</table>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 016 - Time element

Goal: Use time element as a small, standards-first building block.

```
<time datetime="2026-08-07">August 7, 2026</time>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 017 - Machine-readable data

Goal: Use machine-readable data as a small, standards-first building block.

```
<data value="SKU-2048">Blue travel mug</data>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 018 - Inline abbreviation

Goal: Use inline abbreviation as a small, standards-first building block.

```
<p><abbr title="Cascading Style Sheets">CSS</abbr> controls presentation.</p>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 019 - Inline code

Goal: Use inline code as a small, standards-first building block.

`<p>Run npm run dev to start the local server.</p>`

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 020 - Preformatted code sample

Goal: Use preformatted code sample as a small, standards-first building block.

```
<pre><code>const greeting = 'Hello';  
console.log(greeting);</code></pre>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 021 - Details disclosure

Goal: Use details disclosure as a small, standards-first building block.

```
<details>
  <summary>What is included?</summary>
  <p>HTML, CSS, JavaScript, and testing notes.</p>
</details>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 022 - Dialog element

Goal: Use dialog element as a small, standards-first building block.

```
<dialog id="confirm-dialog">
  <form method="dialog">
    <p>Delete this draft?</p>
    <button value="cancel">Cancel</button>
    <button value="confirm">Delete</button>
  </form>
</dialog>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 023 - Progress indicator

Goal: Use progress indicator as a small, standards-first building block.

```
<label for="upload">Upload progress</label>
<progress id="upload" max="100" value="62">62%</progress>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 024 - Meter gauge

Goal: Use meter gauge as a small, standards-first building block.

```
<label for="score">Quality score</label>
<meter id="score" min="0" max="100" low="40" high="80" optimum="100" value="88">88/100</meter>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 025 - Marked text

Goal: Use marked text as a small, standards-first building block.

```
<p>Search result: learn <mark>CSS Grid</mark> visually.</p>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 026 - Block quotation

Goal: Use block quotation as a small, standards-first building block.

```
<figure>
  <blockquote cite="https://example.com/source"><p>A short quotation belongs
    here.</p></blockquote>
  <figcaption>- Source name</figcaption>
</figure>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 027 - Contact address

Goal: Use contact address as a small, standards-first building block.

```
<address>
  Questions? <a href="mailto:help@example.com">help@example.com</a>
</address>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 028 - Basic labeled text field

Goal: Use basic labeled text field as a small, standards-first building block.

```
<label for="display-name">Display name</label>
<input id="display-name" name="displayName" autocomplete="name">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 029 - Required email field

Goal: Use required email field as a small, standards-first building block.

```
<label for="email">Email</label>  
<input id="email" name="email" type="email" autocomplete="email" required>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 030 - Password field with hint

Goal: Use password field with hint as a small, standards-first building block.

```
<label for="password">Password</label>
<p id="password-hint">Use at least 12 characters.</p>
<input id="password" type="password" minlength="12" autocomplete="new-password" aria-
describedby="password-hint" required>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 031 - Search field

Goal: Use search field as a small, standards-first building block.

```
<form role="search" action="/search">
  <label for="q">Search</label>
  <input id="q" name="q" type="search">
  <button>Search</button>
</form>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 032 - Telephone field

Goal: Use telephone field as a small, standards-first building block.

```
<label for="phone">Phone</label>
<input id="phone" name="phone" type="tel" autocomplete="tel" inputmode="tel">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 033 - Numeric field

Goal: Use numeric field as a small, standards-first building block.

```
<label for="quantity">Quantity</label>
<input id="quantity" name="quantity" type="number" min="1" max="20" step="1" value="1">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 034 - Date field

Goal: Use date field as a small, standards-first building block.

```
<label for="start">Start date</label>  
<input id="start" name="start" type="date">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 035 - Color field

Goal: Use color field as a small, standards-first building block.

```
<label for="accent">Accent color</label>
<input id="accent" name="accent" type="color" value="#6d5dfc">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 036 - Range field with output

Goal: Use range field with output as a small, standards-first building block.

```
<label for="volume">Volume</label>
<input id="volume" type="range" min="0" max="100" value="50">
<output for="volume">50</output>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 037 - Textarea

Goal: Use textarea as a small, standards-first building block.

```
<label for="bio">Short bio</label>  
<textarea id="bio" name="bio" rows="5" maxlength="300"></textarea>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 038 - Select menu

Goal: Use select menu as a small, standards-first building block.

```
<label for="theme">Theme</label>
<select id="theme" name="theme">
  <option value="system">System</option>
  <option value="light">Light</option>
  <option value="dark">Dark</option>
</select>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 039 - Select groups

Goal: Use select groups as a small, standards-first building block.

```
<label for="city">City</label>
<select id="city">
  <optgroup label="Americas"><option>Toronto</option><option>Lima</option></optgroup>
  <optgroup label="Europe"><option>Lisbon</option><option>Oslo</option></optgroup>
</select>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 040 - Datalist suggestions

Goal: Use datalist suggestions as a small, standards-first building block.

```
<label for="framework">Framework</label>
<input id="framework" list="frameworks">
<datalist id="frameworks"><option value="React"><option value="Vue"><option
  value="Svelte"></datalist>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 041 - Checkbox group

Goal: Use checkbox group as a small, standards-first building block.

```
<fieldset>
  <legend>Notifications</legend>
  <label><input type="checkbox" name="notify" value="email"> Email</label>
  <label><input type="checkbox" name="notify" value="push"> Push</label>
</fieldset>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 042 - Radio group

Goal: Use radio group as a small, standards-first building block.

```
<fieldset>
  <legend>Billing cycle</legend>
  <label><input type="radio" name="cycle" value="monthly" checked> Monthly</label>
  <label><input type="radio" name="cycle" value="yearly"> Yearly</label>
</fieldset>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 043 - File upload

Goal: Use file upload as a small, standards-first building block.

```
<label for="avatar">Profile photo</label>  
<input id="avatar" name="avatar" type="file" accept="image/png,image/jpeg">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 044 - Form fieldset

Goal: Use form fieldset as a small, standards-first building block.

```
<fieldset>
  <legend>Shipping address</legend>
  <label>Street <input name="street" autocomplete="shipping street-address"></label>
  <label>Postal code <input name="postal" autocomplete="shipping postal-code"></label>
</fieldset>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 045 - Native form validation

Goal: Use native form validation as a small, standards-first building block.

```
<form>
  <label>Username <input name="username" minlength="3" maxlength="24" pattern="[A-Za-z0-9_]+"
    required></label>
  <button>Continue</button>
</form>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 046 - Autocomplete tokens

Goal: Use autocomplete tokens as a small, standards-first building block.

```
<label>Full name <input autocomplete="name"></label>
<label>Organization <input autocomplete="organization"></label>
<label>Country <input autocomplete="country-name"></label>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 047 - Button types

Goal: Use button types as a small, standards-first building block.

```
<form>
  <button type="submit">Save</button>
  <button type="reset">Reset form</button>
  <button type="button" id="preview">Preview</button>
</form>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 048 - Disabled field

Goal: Use disabled field as a small, standards-first building block.

```
<label>Account ID <input value="A-2048" disabled></label>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 049 - Readonly field

Goal: Use readonly field as a small, standards-first building block.

```
<label>Referral code <input value="HELL026" readonly></label>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 050 - Hidden input

Goal: Use hidden input as a small, standards-first building block.

```
<input type="hidden" name="csrf_token" value="SERVER_GENERATED_TOKEN">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 051 - Accessible iframe title

Goal: Use accessible iframe title as a small, standards-first building block.

```
<iframe src="/demo/map.html" title="Map showing pickup locations" loading="lazy"></iframe>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 052 - Lazy video

Goal: Use lazy video as a small, standards-first building block.

```
<video controls preload="metadata" width="720" poster="poster.webp">
  <source src="demo.webm" type="video/webm">
  <track kind="captions" src="captions.vtt" srclang="en" label="English" default>
</video>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 053 - Audio with fallback

Goal: Use audio with fallback as a small, standards-first building block.

```
<audio controls preload="metadata">
  <source src="lesson.ogg" type="audio/ogg">
  <source src="lesson.mp3" type="audio/mpeg">
  <a href="lesson.mp3">Download the audio</a>
</audio>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 054 - Accessible inline SVG

Goal: Use accessible inline svg as a small, standards-first building block.

```
<svg viewBox="0 0 24 24" role="img" aria-labelledby="check-title">
  <title id="check-title">Success</title>
  <path d="M5 12l4 4L19 6" fill="none" stroke="currentColor"/>
</svg>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 055 - Decorative inline SVG

Goal: Use decorative inline svg as a small, standards-first building block.

```
<svg viewBox="0 0 24 24" aria-hidden="true" focusable="false"><circle cx="12" cy="12"
  r="10"/></svg>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 056 - Canvas fallback

Goal: Use canvas fallback as a small, standards-first building block.

```
<canvas id="chart" width="640" height="360">
  Monthly sales chart. <a href="sales.csv">Download the data</a>.
</canvas>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 057 - Template element

Goal: Use template element as a small, standards-first building block.

```
<template id="card-template">
  <article class="card"><h2></h2><p></p></article>
</template>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 058 - Data attributes

Goal: Use data attributes as a small, standards-first building block.

```
<button data-action="favorite" data-item-id="42">Favorite</button>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 059 - Noscript fallback

Goal: Use noscript fallback as a small, standards-first building block.

```
<noscript><p>This calculator needs JavaScript. You can use the <a href="/manual">manual  
worksheet</a> instead.</p></noscript>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 060 - Canonical URL

Goal: Use canonical url as a small, standards-first building block.

```
<link rel="canonical" href="https://example.com/guides/css-grid">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 061 - Theme color metadata

Goal: Use theme color metadata as a small, standards-first building block.

```
<meta name="theme-color" content="#111827" media="(prefers-color-scheme: dark)">
<meta name="theme-color" content="#ffffff" media="(prefers-color-scheme: light)">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 062 - Social sharing metadata

Goal: Use social sharing metadata as a small, standards-first building block.

```
<meta property="og:title" content="Practical CSS Grid Guide">
<meta property="og:description" content="Build responsive grids step by step.">
<meta property="og:image" content="https://example.com/social/grid-guide.png">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 063 - Preconnect hint

Goal: Use preconnect hint as a small, standards-first building block.

```
<link rel="preconnect" href="https://fonts.example-cdn.com" crossorigin>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 064 - Preload critical font

Goal: Use preload critical font as a small, standards-first building block.

```
<link rel="preload" href="/fonts/site.woff2" as="font" type="font/woff2" crossorigin>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 065 - Module script

Goal: Use module script as a small, standards-first building block.

```
<script type="module" src="/js/app.js"></script>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 066 - Deferred classic script

Goal: Use deferred classic script as a small, standards-first building block.

```
<script src="/js/legacy.js" defer></script>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 067 - PWA manifest link

Goal: Use pwa manifest link as a small, standards-first building block.

```
<link rel="manifest" href="/site.webmanifest">
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 068 - Download link

Goal: Use download link as a small, standards-first building block.

```
<a href="/files/worksheet.pdf" download>Download the worksheet (PDF)</a>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 069 - Language switch link

Goal: Use language switch link as a small, standards-first building block.

```
<a href="/es/guia" lang="es" hreflang="es">Español</a>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 070 - Bi-directional isolation

Goal: Use bi-directional isolation as a small, standards-first building block.

```
<p>Winner: <bdi>مَدْخُوتِ سَمِ الْجَمْسَا</bdi> - 128 points</p>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 071 - Keyboard text

Goal: Use keyboard text as a small, standards-first building block.

```
<p>Press <kbd>Ctrl</kbd> + <kbd>K</kbd> to open search.</p>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

Snippet 072 - Sample output

Goal: Use sample output as a small, standards-first building block.

```
<p>Expected terminal output: <samp>Server ready on :3000</samp></p>
```

Step by step

1. Paste the snippet into the document where its meaning belongs.
2. Replace placeholder text/URLs with real content; keep semantic relationships such as labels, headings, and captions intact.
3. Inspect the rendered DOM and use only attributes that communicate real state or purpose.
4. Test with keyboard navigation and a narrow viewport before styling it heavily.

Verify: Reload with DevTools open; verify the element is present, keyboard reachable when interactive, and announced sensibly by accessibility inspection tools.

Gotcha: HTML semantics are behavior, not decoration. Do not add ARIA that contradicts native element behavior, and never put untrusted HTML directly into the page.

CSS micro-recipes

Recipes 073-222

Snippet 073 - Universal border-box sizing - core

Goal: Apply the smallest useful version of universal border-box sizing.

```
.universal-border-box-sizing {  
  box-sizing: border-box;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 074 - Universal border-box sizing - responsive opt-in

Goal: Introduce universal border-box sizing only when at least 48rem of viewport width is available.

```
.universal-border-box-sizing {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .universal-border-box-sizing {  
    box-sizing: border-box;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 075 - Reset body margin - core

Goal: Apply the smallest useful version of reset body margin.

```
.reset-body-margin {  
  margin: 0;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 076 - Reset body margin - responsive opt-in

Goal: Introduce reset body margin only when at least 48rem of viewport width is available.

```
.reset-body-margin {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .reset-body-margin {  
    margin: 0;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 077 - Readable line length - core

Goal: Apply the smallest useful version of readable line length.

```
.readable-line-length {  
  max-inline-size: 68ch;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 078 - Readable line length - responsive opt-in

Goal: Introduce readable line length only when at least 48rem of viewport width is available.

```
.readable-line-length {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .readable-line-length {  
    max-inline-size: 68ch;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 079 - Fluid page gutter - core

Goal: Apply the smallest useful version of fluid page gutter.

```
.fluid-page-gutter {  
  padding-inline: clamp(1rem, 4vw, 4rem);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 080 - Fluid page gutter - responsive opt-in

Goal: Introduce fluid page gutter only when at least 48rem of viewport width is available.

```
.fluid-page-gutter {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .fluid-page-gutter {  
    padding-inline: clamp(1rem, 4vw, 4rem);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 081 - Horizontal auto centering - core

Goal: Apply the smallest useful version of horizontal auto centering.

```
.horizontal-auto-centering {  
  margin-inline: auto;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 082 - Horizontal auto centering - responsive opt-in

Goal: Introduce horizontal auto centering only when at least 48rem of viewport width is available.

```
.horizontal-auto-centering {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .horizontal-auto-centering {  
    margin-inline: auto;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 083 - Grid super centering - core

Goal: Apply the smallest useful version of grid super centering.

```
.grid-super-centering {  
  display: grid;  
  place-items: center;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 084 - Grid super centering - responsive opt-in

Goal: Introduce grid super centering only when at least 48rem of viewport width is available.

```
.grid-super-centering {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .grid-super-centering {  
    display: grid;  
    place-items: center;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 085 - Flex super centering - core

Goal: Apply the smallest useful version of flex super centering.

```
.flex-super-centering {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 086 - Flex super centering - responsive opt-in

Goal: Introduce flex super centering only when at least 48rem of viewport width is available.

```
.flex-super-centering {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .flex-super-centering {  
    display: flex;  
    align-items: center;  
    justify-content: center;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 087 - Flex row with gap - core

Goal: Apply the smallest useful version of flex row with gap.

```
.flex-row-with-gap {  
  display: flex;  
  align-items: center;  
  gap: 1rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 088 - Flex row with gap - responsive opt-in

Goal: Introduce flex row with gap only when at least 48rem of viewport width is available.

```
.flex-row-with-gap {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .flex-row-with-gap {  
    display: flex;  
    align-items: center;  
    gap: 1rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 089 - Flex wrap chips - core

Goal: Apply the smallest useful version of flex wrap chips.

```
.flex-wrap-chips {  
  display: flex;  
  flex-wrap: wrap;  
  gap: .5rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 090 - Flex wrap chips - responsive opt-in

Goal: Introduce flex wrap chips only when at least 48rem of viewport width is available.

```
.flex-wrap-chips {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .flex-wrap-chips {  
    display: flex;  
    flex-wrap: wrap;  
    gap: .5rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 091 - Push final flex item - core

Goal: Apply the smallest useful version of push final flex item.

```
.push-final-flex-item {  
  display: flex;  
  .last { margin-inline-start: auto; }  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 092 - Push final flex item - responsive opt-in

Goal: Introduce push final flex item only when at least 48rem of viewport width is available.

```
.push-final-flex-item {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .push-final-flex-item {  
    display: flex;  
    .last { margin-inline-start: auto; }  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 093 - Equal-width flex children - core

Goal: Apply the smallest useful version of equal-width flex children.

```
.equal-width-flex-children {  
  display: flex;  
  > * { flex: 1 1 0; }  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 094 - Equal-width flex children - responsive opt-in

Goal: Introduce equal-width flex children only when at least 48rem of viewport width is available.

```
.equal-width-flex-children {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .equal-width-flex-children {  
    display: flex;  
    > * { flex: 1 1 0; }  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 095 - Auto-fit grid - core

Goal: Apply the smallest useful version of auto-fit grid.

```
.auto-fit-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr));  
  gap: 1rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 096 - Auto-fit grid - responsive opt-in

Goal: Introduce auto-fit grid only when at least 48rem of viewport width is available.

```
.auto-fit-grid {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .auto-fit-grid {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr));  
    gap: 1rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 097 - Twelve-column grid - core

Goal: Apply the smallest useful version of twelve-column grid.

```
.twelve-column-grid {  
  display: grid;  
  grid-template-columns: repeat(12, minmax(0, 1fr));  
  gap: 1rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 098 - Twelve-column grid - responsive opt-in

Goal: Introduce twelve-column grid only when at least 48rem of viewport width is available.

```
.twelve-column-grid {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .twelve-column-grid {  
    display: grid;  
    grid-template-columns: repeat(12, minmax(0, 1fr));  
    gap: 1rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 099 - Named grid areas - core

Goal: Apply the smallest useful version of named grid areas.

```
.named-grid-areas {  
  display: grid;  
  grid-template-areas: 'header header' 'nav main' 'footer footer';  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 100 - Named grid areas - responsive opt-in

Goal: Introduce named grid areas only when at least 48rem of viewport width is available.

```
.named-grid-areas {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .named-grid-areas {  
    display: grid;  
    grid-template-areas: 'header header' 'nav main' 'footer footer';  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 101 - Intrinsic sidebar layout - core

Goal: Apply the smallest useful version of intrinsic sidebar layout.

```
.intrinsic-sidebar-layout {  
  display: grid;  
  grid-template-columns: minmax(12rem, 18rem) minmax(0, 1fr);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 102 - Intrinsic sidebar layout - responsive opt-in

Goal: Introduce intrinsic sidebar layout only when at least 48rem of viewport width is available.

```
.intrinsic-sidebar-layout {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .intrinsic-sidebar-layout {  
    display: grid;  
    grid-template-columns: minmax(12rem, 18rem) minmax(0, 1fr);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 103 - Responsive RAM grid - core

Goal: Apply the smallest useful version of responsive ram grid.

```
.responsive-ram-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(18rem, 1fr));  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 104 - Responsive RAM grid - responsive opt-in

Goal: Introduce responsive ram grid only when at least 48rem of viewport width is available.

```
.responsive-ram-grid {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .responsive-ram-grid {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(18rem, 1fr));  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 105 - Sticky header - core

Goal: Apply the smallest useful version of sticky header.

```
.sticky-header {  
  position: sticky;  
  top: 0;  
  z-index: 10;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 106 - Sticky header - responsive opt-in

Goal: Introduce sticky header only when at least 48rem of viewport width is available.

```
.sticky-header {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .sticky-header {  
    position: sticky;  
    top: 0;  
    z-index: 10;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 107 - Fixed action button - core

Goal: Apply the smallest useful version of fixed action button.

```
.fixed-action-button {  
  position: fixed;  
  right: 1rem;  
  bottom: 1rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 108 - Fixed action button - responsive opt-in

Goal: Introduce fixed action button only when at least 48rem of viewport width is available.

```
.fixed-action-button {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .fixed-action-button {  
    position: fixed;  
    right: 1rem;  
    bottom: 1rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 109 - Absolute overlay - core

Goal: Apply the smallest useful version of absolute overlay.

```
.absolute-overlay {  
  position: absolute;  
  inset: 0;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 110 - Absolute overlay - responsive opt-in

Goal: Introduce absolute overlay only when at least 48rem of viewport width is available.

```
.absolute-overlay {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .absolute-overlay {  
    position: absolute;  
    inset: 0;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 111 - Full viewport section - core

Goal: Apply the smallest useful version of full viewport section.

```
.full-viewport-section {  
  min-block-size: 100dvh;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 112 - Full viewport section - responsive opt-in

Goal: Introduce full viewport section only when at least 48rem of viewport width is available.

```
.full-viewport-section {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .full-viewport-section {  
    min-block-size: 100dvh;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 113 - Safe viewport section - core

Goal: Apply the smallest useful version of safe viewport section.

```
.safe-viewport-section {  
  min-block-size: 100svh;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 114 - Safe viewport section - responsive opt-in

Goal: Introduce safe viewport section only when at least 48rem of viewport width is available.

```
.safe-viewport-section {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .safe-viewport-section {  
    min-block-size: 100svh;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 115 - Aspect-ratio card - core

Goal: Apply the smallest useful version of aspect-ratio card.

```
.aspect-ratio-card {  
  aspect-ratio: 16 / 9;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 116 - Aspect-ratio card - responsive opt-in

Goal: Introduce aspect-ratio card only when at least 48rem of viewport width is available.

```
.aspect-ratio-card {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .aspect-ratio-card {  
    aspect-ratio: 16 / 9;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 117 - Square media - core

Goal: Apply the smallest useful version of square media.

```
.square-media {  
  aspect-ratio: 1;  
  object-fit: cover;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 118 - Square media - responsive opt-in

Goal: Introduce square media only when at least 48rem of viewport width is available.

```
.square-media {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .square-media {  
    aspect-ratio: 1;  
    object-fit: cover;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 119 - Cover image - core

Goal: Apply the smallest useful version of cover image.

```
.cover-image {  
  inline-size: 100%;  
  block-size: 100%;  
  object-fit: cover;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 120 - Cover image - responsive opt-in

Goal: Introduce cover image only when at least 48rem of viewport width is available.

```
.cover-image {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .cover-image {  
    inline-size: 100%;  
    block-size: 100%;  
    object-fit: cover;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 121 - Contain image - core

Goal: Apply the smallest useful version of contain image.

```
.contain-image {  
  max-inline-size: 100%;  
  block-size: auto;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 122 - Contain image - responsive opt-in

Goal: Introduce contain image only when at least 48rem of viewport width is available.

```
.contain-image {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .contain-image {  
    max-inline-size: 100%;  
    block-size: auto;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 123 - Horizontal scroller - core

Goal: Apply the smallest useful version of horizontal scroller.

```
.horizontal-scroller {  
  display: grid;  
  grid-auto-flow: column;  
  grid-auto-columns: min(80%, 22rem);  
  overflow-x: auto;  
  scroll-snap-type: x mandatory;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 124 - Horizontal scroller - responsive opt-in

Goal: Introduce horizontal scroller only when at least 48rem of viewport width is available.

```
.horizontal-scroller {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .horizontal-scroller {  
    display: grid;  
    grid-auto-flow: column;  
    grid-auto-columns: min(80%, 22rem);  
    overflow-x: auto;  
    scroll-snap-type: x mandatory;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 125 - Scroll snap child - core

Goal: Apply the smallest useful version of scroll snap child.

```
.scroll-snap-child {  
  scroll-snap-align: start;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 126 - Scroll snap child - responsive opt-in

Goal: Introduce scroll snap child only when at least 48rem of viewport width is available.

```
.scroll-snap-child {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .scroll-snap-child {  
    scroll-snap-align: start;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 127 - Hide overflow - core

Goal: Apply the smallest useful version of hide overflow.

```
.hide-overflow {  
  overflow: hidden;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 128 - Hide overflow - responsive opt-in

Goal: Introduce hide overflow only when at least 48rem of viewport width is available.

```
.hide-overflow {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .hide-overflow {  
    overflow: hidden;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 129 - Auto overflow - core

Goal: Apply the smallest useful version of auto overflow.

```
.auto-overflow {  
  overflow: auto;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 130 - Auto overflow - responsive opt-in

Goal: Introduce auto overflow only when at least 48rem of viewport width is available.

```
.auto-overflow {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .auto-overflow {  
    overflow: auto;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 131 - Stable scrollbar gutter - core

Goal: Apply the smallest useful version of stable scrollbar gutter.

```
.stable-scrollbar-gutter {  
  scrollbar-gutter: stable;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 132 - Stable scrollbar gutter - responsive opt-in

Goal: Introduce stable scrollbar gutter only when at least 48rem of viewport width is available.

```
.stable-scrollbar-gutter {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .stable-scrollbar-gutter {  
    scrollbar-gutter: stable;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 133 - Single-line ellipsis - core

Goal: Apply the smallest useful version of single-line ellipsis.

```
.single-line-ellipsis {  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 134 - Single-line ellipsis - responsive opt-in

Goal: Introduce single-line ellipsis only when at least 48rem of viewport width is available.

```
.single-line-ellipsis {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .single-line-ellipsis {  
    overflow: hidden;  
    text-overflow: ellipsis;  
    white-space: nowrap;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 135 - Multi-line clamp - core

Goal: Apply the smallest useful version of multi-line clamp.

```
.multi-line-clamp {  
  display: -webkit-box;  
  -webkit-line-clamp: 3;  
  -webkit-box-orient: vertical;  
  overflow: hidden;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 136 - Multi-line clamp - responsive opt-in

Goal: Introduce multi-line clamp only when at least 48rem of viewport width is available.

```
.multi-line-clamp {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .multi-line-clamp {  
    display: -webkit-box;  
    -webkit-line-clamp: 3;  
    -webkit-box-orient: vertical;  
    overflow: hidden;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 137 - Safe long-word wrapping - core

Goal: Apply the smallest useful version of safe long-word wrapping.

```
.safe-long-word-wrapping {  
  overflow-wrap: anywhere;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 138 - Safe long-word wrapping - responsive opt-in

Goal: Introduce safe long-word wrapping only when at least 48rem of viewport width is available.

```
.safe-long-word-wrapping {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .safe-long-word-wrapping {  
    overflow-wrap: anywhere;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 139 - Balanced heading - core

Goal: Apply the smallest useful version of balanced heading.

```
.balanced-heading {  
  text-wrap: balance;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 140 - Balanced heading - responsive opt-in

Goal: Introduce balanced heading only when at least 48rem of viewport width is available.

```
.balanced-heading {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .balanced-heading {  
    text-wrap: balance;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 141 - Pretty paragraph wrapping - core

Goal: Apply the smallest useful version of pretty paragraph wrapping.

```
.pretty-paragraph-wrapping {  
  text-wrap: pretty;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 142 - Pretty paragraph wrapping - responsive opt-in

Goal: Introduce pretty paragraph wrapping only when at least 48rem of viewport width is available.

```
.pretty-paragraph-wrapping {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .pretty-paragraph-wrapping {  
    text-wrap: pretty;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 143 - Fluid heading size - core

Goal: Apply the smallest useful version of fluid heading size.

```
.fluid-heading-size {  
  font-size: clamp(2rem, 5vw, 5rem);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 144 - Fluid heading size - responsive opt-in

Goal: Introduce fluid heading size only when at least 48rem of viewport width is available.

```
.fluid-heading-size {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .fluid-heading-size {  
    font-size: clamp(2rem, 5vw, 5rem);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 145 - System font stack - core

Goal: Apply the smallest useful version of system font stack.

```
.system-font-stack {  
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 146 - System font stack - responsive opt-in

Goal: Introduce system font stack only when at least 48rem of viewport width is available.

```
.system-font-stack {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .system-font-stack {  
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 147 - Tabular numerals - core

Goal: Apply the smallest useful version of tabular numerals.

```
.tabular-numerals {  
  font-variant-numeric: tabular-nums;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 148 - Tabular numerals - responsive opt-in

Goal: Introduce tabular numerals only when at least 48rem of viewport width is available.

```
.tabular-numerals {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .tabular-numerals {  
    font-variant-numeric: tabular-nums;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 149 - Uppercase label tracking - core

Goal: Apply the smallest useful version of uppercase label tracking.

```
.uppercase-label-tracking {  
  text-transform: uppercase;  
  letter-spacing: .08em;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 150 - Uppercase label tracking - responsive opt-in

Goal: Introduce uppercase label tracking only when at least 48rem of viewport width is available.

```
.uppercase-label-tracking {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .uppercase-label-tracking {  
    text-transform: uppercase;  
    letter-spacing: .08em;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 151 - Underline tuning - core

Goal: Apply the smallest useful version of underline tuning.

```
.underline-tuning {  
  text-decoration-thickness: .08em;  
  text-underline-offset: .2em;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 152 - Underline tuning - responsive opt-in

Goal: Introduce underline tuning only when at least 48rem of viewport width is available.

```
.underline-tuning {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .underline-tuning {  
    text-decoration-thickness: .08em;  
    text-underline-offset: .2em;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 153 - Current-color icon - core

Goal: Apply the smallest useful version of current-color icon.

```
.current-color-icon {  
  fill: none;  
  stroke: currentColor;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 154 - Current-color icon - responsive opt-in

Goal: Introduce current-color icon only when at least 48rem of viewport width is available.

```
.current-color-icon {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .current-color-icon {  
    fill: none;  
    stroke: currentColor;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 155 - Design token colors - core

Goal: Apply the smallest useful version of design token colors.

```
.design-token-colors {  
  color: var(--text, #111827);  
  background: var(--surface, #ffffff);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 156 - Design token colors - responsive opt-in

Goal: Introduce design token colors only when at least 48rem of viewport width is available.

```
.design-token-colors {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .design-token-colors {  
    color: var(--text, #111827);  
    background: var(--surface, #ffffff);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 157 - Light-dark color - core

Goal: Apply the smallest useful version of light-dark color.

```
.light-dark-color {  
  color-scheme: light dark;  
  background: light-dark(#fff, #111827);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 158 - Light-dark color - responsive opt-in

Goal: Introduce light-dark color only when at least 48rem of viewport width is available.

```
.light-dark-color {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .light-dark-color {  
    color-scheme: light dark;  
    background: light-dark(#fff, #111827);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 159 - Linear gradient - core

Goal: Apply the smallest useful version of linear gradient.

```
.linear-gradient {  
  background: linear-gradient(135deg, #7c3aed, #06b6d4);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 160 - Linear gradient - responsive opt-in

Goal: Introduce linear gradient only when at least 48rem of viewport width is available.

```
.linear-gradient {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .linear-gradient {  
    background: linear-gradient(135deg, #7c3aed, #06b6d4);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 161 - Radial glow - core

Goal: Apply the smallest useful version of radial glow.

```
.radial-glow {  
  background: radial-gradient(circle at 20% 20%, #60a5fa, transparent 45%);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 162 - Radial glow - responsive opt-in

Goal: Introduce radial glow only when at least 48rem of viewport width is available.

```
.radial-glow {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .radial-glow {  
    background: radial-gradient(circle at 20% 20%, #60a5fa, transparent 45%);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 163 - Conic gradient - core

Goal: Apply the smallest useful version of conic gradient.

```
.conic-gradient {  
  background: conic-gradient(from 45deg, #f43f5e, #f59e0b, #22c55e, #3b82f6, #f43f5e);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 164 - Conic gradient - responsive opt-in

Goal: Introduce conic gradient only when at least 48rem of viewport width is available.

```
.conic-gradient {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .conic-gradient {  
    background: conic-gradient(from 45deg, #f43f5e, #f59e0b, #22c55e, #3b82f6, #f43f5e);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 165 - Grid-paper background - core

Goal: Apply the smallest useful version of grid-paper background.

```
.grid-paper-background {  
  background-image: linear-gradient(#000 1px, transparent 1px), linear-gradient(90deg, #000  
    1px, transparent 1px);  
  background-size: 24px 24px;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 166 - Grid-paper background - responsive opt-in

Goal: Introduce grid-paper background only when at least 48rem of viewport width is available.

```
.grid-paper-background {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .grid-paper-background {  
    background-image: linear-gradient(#0001 1px, transparent 1px), linear-gradient(90deg,  
      #0001 1px, transparent 1px);  
    background-size: 24px 24px;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 167 - Soft card shadow - core

Goal: Apply the smallest useful version of soft card shadow.

```
.soft-card-shadow {  
  box-shadow: 0 12px 32px rgb(15 23 42 / .12);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 168 - Soft card shadow - responsive opt-in

Goal: Introduce soft card shadow only when at least 48rem of viewport width is available.

```
.soft-card-shadow {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .soft-card-shadow {  
    box-shadow: 0 12px 32px rgb(15 23 42 / .12);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 169 - Layered shadow - core

Goal: Apply the smallest useful version of layered shadow.

```
.layered-shadow {  
  box-shadow: 0 1px 2px #0002, 0 8px 24px #0002, 0 24px 64px #0001;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 170 - Layered shadow - responsive opt-in

Goal: Introduce layered shadow only when at least 48rem of viewport width is available.

```
.layered-shadow {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .layered-shadow {  
    box-shadow: 0 1px 2px #0002, 0 8px 24px #0002, 0 24px 64px #0001;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 171 - Inset border - core

Goal: Apply the smallest useful version of inset border.

```
.inset-border {  
  box-shadow: inset 0 0 0 1px rgb(255 255 255 / .16);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 172 - Inset border - responsive opt-in

Goal: Introduce inset border only when at least 48rem of viewport width is available.

```
.inset-border {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .inset-border {  
    box-shadow: inset 0 0 0 1px rgb(255 255 255 / .16);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 173 - Rounded card - core

Goal: Apply the smallest useful version of rounded card.

```
.rounded-card {  
  border-radius: 1rem;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 174 - Rounded card - responsive opt-in

Goal: Introduce rounded card only when at least 48rem of viewport width is available.

```
.rounded-card {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .rounded-card {  
    border-radius: 1rem;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 175 - Pill shape - core

Goal: Apply the smallest useful version of pill shape.

```
.pill-shape {  
  border-radius: 999px;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 176 - Pill shape - responsive opt-in

Goal: Introduce pill shape only when at least 48rem of viewport width is available.

```
.pill-shape {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .pill-shape {  
    border-radius: 999px;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 177 - Circular avatar - core

Goal: Apply the smallest useful version of circular avatar.

```
.circular-avatar {  
  aspect-ratio: 1;  
  border-radius: 50%;  
  object-fit: cover;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 178 - Circular avatar - responsive opt-in

Goal: Introduce circular avatar only when at least 48rem of viewport width is available.

```
.circular-avatar {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .circular-avatar {  
    aspect-ratio: 1;  
    border-radius: 50%;  
    object-fit: cover;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 179 - Glass panel - core

Goal: Apply the smallest useful version of glass panel.

```
.glass-panel {  
  background: rgb(255 255 255 / .12);  
  backdrop-filter: blur(16px);  
  border: 1px solid rgb(255 255 255 / .22);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 180 - Glass panel - responsive opt-in

Goal: Introduce glass panel only when at least 48rem of viewport width is available.

```
.glass-panel {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .glass-panel {  
    background: rgb(255 255 255 / .12);  
    backdrop-filter: blur(16px);  
    border: 1px solid rgb(255 255 255 / .22);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 181 - Grayscale media - core

Goal: Apply the smallest useful version of grayscale media.

```
.grayscale-media {  
  filter: grayscale(1);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 182 - Grayscale media - responsive opt-in

Goal: Introduce grayscale media only when at least 48rem of viewport width is available.

```
.grayscale-media {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .grayscale-media {  
    filter: grayscale(1);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 183 - Drop shadow alpha graphic - core

Goal: Apply the smallest useful version of drop shadow alpha graphic.

```
.drop-shadow-alpha-graphic {  
  filter: drop-shadow(0 12px 20px rgb(0 0 0 / .25));  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 184 - Drop shadow alpha graphic - responsive opt-in

Goal: Introduce drop shadow alpha graphic only when at least 48rem of viewport width is available.

```
.drop-shadow-alpha-graphic {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .drop-shadow-alpha-graphic {  
    filter: drop-shadow(0 12px 20px rgb(0 0 0 / .25));  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 185 - Blend overlay - core

Goal: Apply the smallest useful version of blend overlay.

```
.blend-overlay {  
  mix-blend-mode: screen;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 186 - Blend overlay - responsive opt-in

Goal: Introduce blend overlay only when at least 48rem of viewport width is available.

```
.blend-overlay {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .blend-overlay {  
    mix-blend-mode: screen;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 187 - Clip hexagon - core

Goal: Apply the smallest useful version of clip hexagon.

```
.clip-hexagon {  
  clip-path: polygon(25% 7%, 75% 7%, 100% 50%, 75% 93%, 25% 93%, 0 50%);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 188 - Clip hexagon - responsive opt-in

Goal: Introduce clip hexagon only when at least 48rem of viewport width is available.

```
.clip-hexagon {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .clip-hexagon {  
    clip-path: polygon(25% 7%, 75% 7%, 100% 50%, 75% 93%, 25% 93%, 0 50%);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 189 - Mask fade - core

Goal: Apply the smallest useful version of mask fade.

```
.mask-fade {  
  mask-image: linear-gradient(to bottom, #000 70%, transparent);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 190 - Mask fade - responsive opt-in

Goal: Introduce mask fade only when at least 48rem of viewport width is available.

```
.mask-fade {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .mask-fade {  
    mask-image: linear-gradient(to bottom, #000 70%, transparent);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 191 - Isolate stacking context - core

Goal: Apply the smallest useful version of isolate stacking context.

```
.isolate-stacking-context {  
  isolation: isolate;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 192 - Isolate stacking context - responsive opt-in

Goal: Introduce isolate stacking context only when at least 48rem of viewport width is available.

```
.isolate-stacking-context {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .isolate-stacking-context {  
    isolation: isolate;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 193 - Contain paint - core

Goal: Apply the smallest useful version of contain paint.

```
.contain-paint {  
  contain: paint;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 194 - Contain paint - responsive opt-in

Goal: Introduce contain paint only when at least 48rem of viewport width is available.

```
.contain-paint {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .contain-paint {  
    contain: paint;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 195 - Content visibility - core

Goal: Apply the smallest useful version of content visibility.

```
.content-visibility {  
  content-visibility: auto;  
  contain-intrinsic-size: auto 500px;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 196 - Content visibility - responsive opt-in

Goal: Introduce content visibility only when at least 48rem of viewport width is available.

```
.content-visibility {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .content-visibility {  
    content-visibility: auto;  
    contain-intrinsic-size: auto 500px;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 197 - Focus-visible ring - core

Goal: Apply the smallest useful version of focus-visible ring.

```
.focus-visible-ring {  
  outline: 3px solid #2563eb;  
  outline-offset: 3px;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 198 - Focus-visible ring - responsive opt-in

Goal: Introduce focus-visible ring only when at least 48rem of viewport width is available.

```
.focus-visible-ring {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .focus-visible-ring {  
    outline: 3px solid #2563eb;  
    outline-offset: 3px;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 199 - Pointer coarse target - core

Goal: Apply the smallest useful version of pointer coarse target.

```
.pointer-coarse-target {  
  min-inline-size: 44px;  
  min-block-size: 44px;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 200 - Pointer coarse target - responsive opt-in

Goal: Introduce pointer coarse target only when at least 48rem of viewport width is available.

```
.pointer-coarse-target {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .pointer-coarse-target {  
    min-inline-size: 44px;  
    min-block-size: 44px;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 201 - Disable selection on control chrome - core

Goal: Apply the smallest useful version of disable selection on control chrome.

```
.disable-selection-on-control-chrome {  
  user-select: none;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 202 - Disable selection on control chrome - responsive opt-in

Goal: Introduce disable selection on control chrome only when at least 48rem of viewport width is available.

```
.disable-selection-on-control-chrome {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .disable-selection-on-control-chrome {  
    user-select: none;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 203 - Touch manipulation - core

Goal: Apply the smallest useful version of touch manipulation.

```
.touch-manipulation {  
  touch-action: manipulation;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 204 - Touch manipulation - responsive opt-in

Goal: Introduce touch manipulation only when at least 48rem of viewport width is available.

```
.touch-manipulation {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .touch-manipulation {  
    touch-action: manipulation;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 205 - Accent form controls - core

Goal: Apply the smallest useful version of accent form controls.

```
.accent-form-controls {  
  accent-color: #7c3aed;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 206 - Accent form controls - responsive opt-in

Goal: Introduce accent form controls only when at least 48rem of viewport width is available.

```
.accent-form-controls {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .accent-form-controls {  
    accent-color: #7c3aed;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 207 - Caret color - core

Goal: Apply the smallest useful version of caret color.

```
.caret-color {  
  caret-color: #7c3aed;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 208 - Caret color - responsive opt-in

Goal: Introduce caret color only when at least 48rem of viewport width is available.

```
.caret-color {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .caret-color {  
    caret-color: #7c3aed;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 209 - Muted helper text color - core

Goal: Apply the smallest useful version of muted helper text color.

```
.muted-helper-text-color {  
  color: rgb(100 116 139 / .8);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 210 - Muted helper text color - responsive opt-in

Goal: Introduce muted helper text color only when at least 48rem of viewport width is available.

```
.muted-helper-text-color {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .muted-helper-text-color {  
    color: rgb(100 116 139 / .8);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 211 - Smooth property transition - core

Goal: Apply the smallest useful version of smooth property transition.

```
.smooth-property-transition {  
  transition: transform 180ms ease, opacity 180ms ease;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 212 - Smooth property transition - responsive opt-in

Goal: Introduce smooth property transition only when at least 48rem of viewport width is available.

```
.smooth-property-transition {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .smooth-property-transition {  
    transition: transform 180ms ease, opacity 180ms ease;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 213 - Hover lift - core

Goal: Apply the smallest useful version of hover lift.

```
.hover-lift {  
  transform: translateY(-4px);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 214 - Hover lift - responsive opt-in

Goal: Introduce hover lift only when at least 48rem of viewport width is available.

```
.hover-lift {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .hover-lift {  
    transform: translateY(-4px);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 215 - GPU-friendly scale - core

Goal: Apply the smallest useful version of gpu-friendly scale.

```
.gpu-friendly-scale {  
  transform: scale(1.03);  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 216 - GPU-friendly scale - responsive opt-in

Goal: Introduce gpu-friendly scale only when at least 48rem of viewport width is available.

```
.gpu-friendly-scale {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .gpu-friendly-scale {  
    transform: scale(1.03);  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 217 - 3D stage perspective - core

Goal: Apply the smallest useful version of 3d stage perspective.

```
.3d-stage-perspective {  
  perspective: 1000px;  
  transform-style: preserve-3d;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 218 - 3D stage perspective - responsive opt-in

Goal: Introduce 3d stage perspective only when at least 48rem of viewport width is available.

```
.3d-stage-perspective {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .3d-stage-perspective {  
    perspective: 1000px;  
    transform-style: preserve-3d;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 219 - Backface hiding - core

Goal: Apply the smallest useful version of backface hiding.

```
.backface-hiding {  
  backface-visibility: hidden;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 220 - Backface hiding - responsive opt-in

Goal: Introduce backface hiding only when at least 48rem of viewport width is available.

```
.backface-hiding {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .backface-hiding {  
    backface-visibility: hidden;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

Snippet 221 - Transform origin - core

Goal: Apply the smallest useful version of transform origin.

```
.transform-origin {  
  transform-origin: center bottom;  
}
```

Step by step

1. Add the class to the element you want to change.
2. Copy the rule into your stylesheet after base/reset styles.
3. Change the values one at a time so you can see which declaration controls the effect.
4. Check at 320px, a tablet width, and a wide desktop width.

Verify: In DevTools, toggle each declaration off and on; confirm the rule changes only the intended box or text.

Gotcha: A micro-rule can interact with parent layout, writing mode, overflow, or user preferences. Test it in context instead of assuming the isolated demo guarantees production behavior.

Snippet 222 - Transform origin - responsive opt-in

Goal: Introduce transform origin only when at least 48rem of viewport width is available.

```
.transform-origin {  
  /* safe/default state first */  
}  
  
@media (min-width: 48rem) {  
  .transform-origin {  
    transform-origin: center bottom;  
  }  
}
```

Step by step

1. Start with a usable small-screen state outside the media query.
2. Place the enhancement inside the min-width query.
3. Resize through the boundary rather than testing only two fixed screenshots.
4. If the behavior depends on component space instead of viewport space, consider a container query instead.

Verify: Drag the responsive preview slowly across 48rem and confirm content never overlaps, clips, or becomes unreachable.

Gotcha: A breakpoint is a content decision, not a device label. If 48rem does not match the content's failure point, move the breakpoint to where the layout actually needs it.

JavaScript browser recipes

Recipes 223-380

Snippet 223 - Select one element

Goal: Implement the browser-side task: select one element.

```
const card = document.querySelector('.card');  
console.log(card);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 224 - Select many elements

Goal: Implement the browser-side task: select many elements.

```
const cards = [...document.querySelectorAll('.card')];
cards.forEach((card) => card.classList.add('ready'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 225 - Find element by ID

Goal: Implement the browser-side task: find element by id.

```
const dialog = document.getElementById('confirm-dialog');  
console.log(dialog);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 226 - Create an element

Goal: Implement the browser-side task: create an element.

```
const badge = document.createElement('span');  
badge.className = 'badge';  
badge.textContent = 'New';
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 227 - Append an element

Goal: Implement the browser-side task: append an element.

```
const list = document.querySelector('.list');
const item = document.createElement('li');
item.textContent = 'Fresh item';
list?.append(item);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 228 - Prepend an element

Goal: Implement the browser-side task: prepend an element.

```
const feed = document.querySelector('.feed');
const note = document.createElement('p');
note.textContent = 'Newest first';
feed?.prepend(note);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 229 - Remove an element

Goal: Implement the browser-side task: remove an element.

```
document.querySelector('.temporary')?.remove();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 230 - Replace an element

Goal: Implement the browser-side task: replace an element.

```
const oldNode = document.querySelector('.old');
const newNode = document.createElement('div');
newNode.textContent = 'Replacement';
oldNode?.replaceWith(newNode);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 231 - Clone a template

Goal: Implement the browser-side task: clone a template.

```
const template = document.querySelector('#card-template');
const clone = template?.content.cloneNode(true);
if (clone) document.querySelector('.grid').append(clone);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 232 - Read text content

Goal: Implement the browser-side task: read text content.

```
const title = document.querySelector('h1')?.textContent.trim() ?? '';
console.log(title);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 233 - Set text safely

Goal: Implement the browser-side task: set text safely.

```
const status = document.querySelector('[data-status]');  
if (status) status.textContent = 'Saved';
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 234 - Set a safe attribute

Goal: Implement the browser-side task: set a safe attribute.

```
const image = document.querySelector('img.preview');
image?.setAttribute('alt', 'Selected product preview');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 235 - Read a data attribute

Goal: Implement the browser-side task: read a data attribute.

```
const button = document.querySelector('[data-item-id]');  
const itemId = button?.dataset.itemId;  
console.log(itemId);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 236 - Write a data attribute

Goal: Implement the browser-side task: write a data attribute.

```
const panel = document.querySelector('.panel');  
if (panel) panel.dataset.state = 'open';
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 237 - Add a class

Goal: Implement the browser-side task: add a class.

```
document.querySelector('.card')?.classList.add('is-active');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 238 - Remove a class

Goal: Implement the browser-side task: remove a class.

```
document.querySelector('.card')?.classList.remove('is-loading');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 239 - Toggle a class

Goal: Implement the browser-side task: toggle a class.

```
document.querySelector('.menu')?.classList.toggle('is-open');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 240 - Toggle class with explicit state

Goal: Implement the browser-side task: toggle class with explicit state.

```
const open = true;
document.querySelector('.menu').classList.toggle('is-open', open);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 241 - Check for a class

Goal: Implement the browser-side task: check for a class.

```
const active = document.querySelector('.tab')?.classList.contains('is-active') ?? false;  
console.log(active);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 242 - Set a CSS custom property

Goal: Implement the browser-side task: set a css custom property.

```
document.documentElement.style.setProperty('--hue', '262');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 243 - Read computed CSS

Goal: Implement the browser-side task: read computed css.

```
const panel = document.querySelector('.panel');  
if (panel) console.log(getComputedStyle(panel).display);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 244 - Click listener

Goal: Implement the browser-side task: click listener.

```
document.querySelector('.save')?.addEventListener('click', () => {  
  console.log('Save requested');  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 245 - Input listener

Goal: Implement the browser-side task: input listener.

```
document.querySelector('#search')?.addEventListener('input', (event) => {  
  console.log(event.currentTarget.value);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 246 - Change listener

Goal: Implement the browser-side task: change listener.

```
document.querySelector('#theme')?.addEventListener('change', (event) => {  
  document.documentElement.dataset.theme = event.currentTarget.value;  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 247 - Submit listener

Goal: Implement the browser-side task: submit listener.

```
document.querySelector('form')?.addEventListener('submit', (event) => {  
  event.preventDefault();  
  console.log(new FormData(event.currentTarget));  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 248 - Keyboard listener

Goal: Implement the browser-side task: keyboard listener.

```
document.addEventListener('keydown', (event) => {  
  if (event.key === 'Escape') console.log('Escape pressed');  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 249 - Pointer listener

Goal: Implement the browser-side task: pointer listener.

```
document.querySelector('.surface')?.addEventListener('pointerdown', (event) => {  
  console.log(event.pointerType, event.clientX, event.clientY);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 250 - Focus listener

Goal: Implement the browser-side task: focus listener.

```
document.querySelector('input')?.addEventListener('focus', (event) => {  
  event.currentTarget.closest('.field')?.classList.add('has-focus');  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 251 - Blur listener

Goal: Implement the browser-side task: blur listener.

```
document.querySelector('input')?.addEventListener('blur', (event) => {  
  event.currentTarget.closest('.field')?.classList.remove('has-focus');  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 252 - Event delegation

Goal: Implement the browser-side task: event delegation.

```
document.querySelector('.list').addEventListener('click', (event) => {  
  const button = event.target.closest('button[data-action]');  
  if (!button) return;  
  console.log(button.dataset.action);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 253 - One-time listener

Goal: Implement the browser-side task: one-time listener.

```
window.addEventListener('pointerdown', () => console.log('First interaction'), { once: true
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 254 - Passive scroll listener

Goal: Implement the browser-side task: passive scroll listener.

```
window.addEventListener('scroll', () => {  
  console.log(window.scrollY);  
}, { passive: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 255 - Abortable listeners

Goal: Implement the browser-side task: abortable listeners.

```
const controller = new AbortController();
window.addEventListener('resize', () => console.log('resize'), { signal: controller.signal });
controller.abort();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 256 - Custom event

Goal: Implement the browser-side task: custom event.

```
const event = new CustomEvent('cart:add', { detail: { id: 42 } });  
document.dispatchEvent(event);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 257 - Listen for custom event

Goal: Implement the browser-side task: listen for custom event.

```
document.addEventListener('cart:add', (event) => {  
  console.log('Added', event.detail.id);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 258 - Open native dialog

Goal: Implement the browser-side task: open native dialog.

```
document.querySelector('#confirm-dialog')?.showModal();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 259 - Close native dialog

Goal: Implement the browser-side task: close native dialog.

```
document.querySelector('#confirm-dialog')?.close('cancel');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 260 - Toggle details element

Goal: Implement the browser-side task: toggle details element.

```
const details = document.querySelector('details');  
if (details) details.open = !details.open;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 261 - Focus an element

Goal: Implement the browser-side task: focus an element.

```
document.querySelector('#email')?.focus();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 262 - Scroll element into view

Goal: Implement the browser-side task: scroll element into view.

```
document.querySelector('#pricing')?.scrollIntoView({ behavior: 'smooth', block: 'start' });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 263 - Read form data

Goal: Implement the browser-side task: read form data.

```
const form = document.querySelector('form');
if (form) {
  const data = Object.fromEntries(new FormData(form));
  console.log(data);
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 264 - Check form validity

Goal: Implement the browser-side task: check form validity.

```
const form = document.querySelector('form');  
if (form && !form.checkValidity()) form.reportValidity();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 265 - Set custom validation message

Goal: Implement the browser-side task: set custom validation message.

```
const field = document.querySelector('#username');
field?.setCustomValidity(field.value.includes(' ') ? 'Spaces are not allowed.' : '');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 266 - Reset a form

Goal: Implement the browser-side task: reset a form.

```
document.querySelector('form')?.reset();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 267 - Disable submit while pending

Goal: Implement the browser-side task: disable submit while pending.

```
const submit = document.querySelector('button[type=submit]');
if (submit) {
  submit.disabled = true;
  submit.textContent = 'Saving...';
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 268 - Mirror range output

Goal: Implement the browser-side task: mirror range output.

```
const range = document.querySelector('#volume');
const output = document.querySelector('output[for=volume]');
range?.addEventListener('input', () => { if (output) output.value = range.value; });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 269 - Toggle password visibility

Goal: Implement the browser-side task: toggle password visibility.

```
const input = document.querySelector('#password');
const button = document.querySelector('[data-show-password]');
button?.addEventListener('click', () => {
  if (input) input.type = input.type === 'password' ? 'text' : 'password';
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 270 - Read URL query parameter

Goal: Implement the browser-side task: read url query parameter.

```
const params = new URLSearchParams(location.search);
const query = params.get('q') ?? '';
console.log(query);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 271 - Write URL query parameter

Goal: Implement the browser-side task: write url query parameter.

```
const url = new URL(location.href);  
url.searchParams.set('page', '2');  
history.pushState({}, '', url);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 272 - Replace history state

Goal: Implement the browser-side task: replace history state.

```
const url = new URL(location.href);
url.searchParams.delete('debug');
history.replaceState({}, '', url);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 273 - Handle back/forward

Goal: Implement the browser-side task: handle back/forward.

```
window.addEventListener('popstate', () => {  
  console.log('URL state changed:', location.href);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 274 - Store a string locally

Goal: Implement the browser-side task: store a string locally.

```
localStorage.setItem('theme', 'dark');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 275 - Read a local string

Goal: Implement the browser-side task: read a local string.

```
const theme = localStorage.getItem('theme') ?? 'system';  
console.log(theme);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 276 - Store JSON locally

Goal: Implement the browser-side task: store json locally.

```
const prefs = { density: 'compact', motion: false };
localStorage.setItem('prefs', JSON.stringify(prefs));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 277 - Parse local JSON safely

Goal: Implement the browser-side task: parse local json safely.

```
let prefs = {};  
try { prefs = JSON.parse(localStorage.getItem('prefs') ?? '{}'); } catch {}  
console.log(prefs);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 278 - Remove local value

Goal: Implement the browser-side task: remove local value.

```
localStorage.removeItem('prefs');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 279 - Use session storage

Goal: Implement the browser-side task: use session storage.

```
sessionStorage.setItem('wizardStep', '2');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 280 - Fetch JSON

Goal: Implement the browser-side task: fetch json.

```
const response = await fetch('/api/items');
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const items = await response.json();
console.log(items);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 281 - Fetch with timeout

Goal: Implement the browser-side task: fetch with timeout.

```
const signal = AbortSignal.timeout(5000);
const response = await fetch('/api/status', { signal });
console.log(await response.json());
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 282 - POST JSON

Goal: Implement the browser-side task: post json.

```
const response = await fetch('/api/items', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Notebook' })
});
if (!response.ok) throw new Error('Save failed');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 283 - Submit FormData

Goal: Implement the browser-side task: submit formdata.

```
const form = document.querySelector('form');
if (form) {
  const response = await fetch(form.action, { method: 'POST', body: new FormData(form) });
  if (!response.ok) throw new Error('Upload failed');
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 284 - Run requests in parallel

Goal: Implement the browser-side task: run requests in parallel.

```
const [profile, projects] = await Promise.all([
  fetch('/api/profile').then((r) => r.json()),
  fetch('/api/projects').then((r) => r.json())
]);
console.log(profile, projects);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 285 - Settle independent promises

Goal: Implement the browser-side task: settle independent promises.

```
const results = await Promise.allSettled([taskA(), taskB(), taskC()]);
results.forEach((result) => console.log(result.status));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 286 - Retry once after failure

Goal: Implement the browser-side task: retry once after failure.

```
async function withRetry(task) {  
  try { return await task(); }  
  catch { return task(); }  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 287 - Sleep with a promise

Goal: Implement the browser-side task: sleep with a promise.

```
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));  
await delay(300);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 288 - Debounce a function

Goal: Implement the browser-side task: debounce a function.

```
function debounce(fn, wait = 200) {  
  let timer;  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args), wait);  
  };  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 289 - Throttle with requestAnimationFrame

Goal: Implement the browser-side task: throttle with requestAnimationFrame.

```
let queued = false;
window.addEventListener('scroll', () => {
  if (queued) return;
  queued = true;
  requestAnimationFrame(() => {
    queued = false;
    console.log(window.scrollY);
  });
}, { passive: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 290 - Schedule idle work

Goal: Implement the browser-side task: schedule idle work.

```
requestIdleCallback?(() => {  
  console.log('Run non-urgent work');  
}, { timeout: 1000 });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 291 - Run after next paint

Goal: Implement the browser-side task: run after next paint.

```
requestAnimationFrame(() => {  
  requestAnimationFrame(() => console.log('Two frames later'));  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 292 - Observe element visibility

Goal: Implement the browser-side task: observe element visibility.

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => entry.target.classList.toggle('visible', entry.isIntersecting));
}, { threshold: 0.25 });
document.querySelectorAll('.reveal').forEach((el) => observer.observe(el));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 293 - Lazy-load on intersection

Goal: Implement the browser-side task: lazy-load on intersection.

```
const observer = new IntersectionObserver([entry], obs => {
  if (!entry.isIntersecting) return;
  entry.target.src = entry.target.dataset.src;
  obs.unobserve(entry.target);
});
document.querySelectorAll('img[data-src]').forEach((img) => observer.observe(img));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 294 - Observe element size

Goal: Implement the browser-side task: observe element size.

```
const observer = new ResizeObserver(([entry]) => {  
  console.log(entry.contentRect.width);  
});  
const panel = document.querySelector('.panel');  
if (panel) observer.observe(panel);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 295 - Observe DOM mutations

Goal: Implement the browser-side task: observe dom mutations.

```
const observer = new MutationObserver((records) => console.log(records.length));
observer.observe(document.body, { childList: true, subtree: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 296 - Detect reduced motion

Goal: Implement the browser-side task: detect reduced motion.

```
const reduceMotion = matchMedia('(prefers-reduced-motion: reduce)').matches;  
console.log({ reduceMotion });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 297 - Watch color scheme

Goal: Implement the browser-side task: watch color scheme.

```
const scheme = matchMedia('(prefers-color-scheme: dark)');  
scheme.addEventListener('change', (event) => console.log(event.matches ? 'dark' : 'light'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 298 - Copy text to clipboard

Goal: Implement the browser-side task: copy text to clipboard.

```
await navigator.clipboard.writeText('Copy-ready text');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 299 - Read clipboard text

Goal: Implement the browser-side task: read clipboard text.

```
const text = await navigator.clipboard.readText();  
console.log(text);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 300 - Share with Web Share API

Goal: Implement the browser-side task: share with web share api.

```
if (navigator.share) {  
  await navigator.share({ title: document.title, url: location.href });  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 301 - Check online state

Goal: Implement the browser-side task: check online state.

```
window.addEventListener('online', () => console.log('Back online'));  
window.addEventListener('offline', () => console.log('Offline'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 302 - Read viewport size

Goal: Implement the browser-side task: read viewport size.

```
const viewport = { width: innerWidth, height: innerHeight };  
console.log(viewport);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 303 - Read element rectangle

Goal: Implement the browser-side task: read element rectangle.

```
const rect = document.querySelector('.card')?.getBoundingClientRect();
console.log(rect?.top, rect?.width);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 304 - Use visual viewport

Goal: Implement the browser-side task: use visual viewport.

```
visualViewport?.addEventListener('resize', () => {  
  console.log(visualViewport.height);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 305 - Fullscreen request

Goal: Implement the browser-side task: fullscreen request.

```
await document.querySelector('.player')?.requestFullscreen();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 306 - Exit fullscreen

Goal: Implement the browser-side task: exit fullscreen.

```
if (document.fullscreenElement) await document.exitFullscreen();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 307 - Create an object URL

Goal: Implement the browser-side task: create an object url.

```
const blob = new Blob(['hello'], { type: 'text/plain' });
const url = URL.createObjectURL(blob);
console.log(url);
URL.revokeObjectURL(url);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 308 - Download generated text

Goal: Implement the browser-side task: download generated text.

```
const blob = new Blob(['name,score\nAda,98'], { type: 'text/csv' });
const link = Object.assign(document.createElement('a'), { href: URL.createObjectURL(blob),
  download: 'scores.csv' });
link.click();
URL.revokeObjectURL(link.href);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 309 - Format a number

Goal: Implement the browser-side task: format a number.

```
const formatted = new Intl.NumberFormat('en-US', { maximumFractionDigits: 1
  }).format(12345.67);
console.log(formatted);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 310 - Format currency

Goal: Implement the browser-side task: format currency.

```
const price = new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD'
}).format(29.9);
console.log(price);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 311 - Format a date

Goal: Implement the browser-side task: format a date.

```
const label = new Intl.DateTimeFormat('en', { dateStyle: 'medium' }).format(new Date());
console.log(label);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 312 - Format relative time

Goal: Implement the browser-side task: format relative time.

```
const formatter = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });  
console.log(formatter.format(-1, 'day'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 313 - Pluralize with Intl

Goal: Implement the browser-side task: pluralize with intl.

```
const rules = new Intl.PluralRules('en');
const count = 2;
console.log(`${count} ${rules.select(count) === 'one' ? 'item' : 'items'}`);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 314 - Sort strings naturally

Goal: Implement the browser-side task: sort strings naturally.

```
const collator = new Intl.Collator(undefined, { numeric: true });
const files = ['file10', 'file2', 'file1'].sort(collator.compare);
console.log(files);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 315 - Escape regex text

Goal: Implement the browser-side task: escape regex text.

```
const escapeRegex = (value) => value.replace(/[\.\*\+\?\$\{\}\|\[\]\\\]/g, '\\$&');
console.log(escapeRegex('price?'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 316 - Deduplicate an array

Goal: Implement the browser-side task: deduplicate an array.

```
const unique = [...new Set(['css', 'html', 'css'])];  
console.log(unique);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 317 - Group array items

Goal: Implement the browser-side task: group array items.

```
const rows = [{ type: 'css' }, { type: 'js' }, { type: 'css' }];  
const groups = Object.groupBy(rows, (row) => row.type);  
console.log(groups);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 318 - Index array by ID

Goal: Implement the browser-side task: index array by id.

```
const items = [{ id: 1, name: 'A' }, { id: 2, name: 'B' }];  
const byId = new Map(items.map((item) => [item.id, item]));  
console.log(byId.get(2));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 319 - Filter an array

Goal: Implement the browser-side task: filter an array.

```
const visible = items.filter((item) => !item.hidden);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 320 - Transform an array

Goal: Implement the browser-side task: transform an array.

```
const names = users.map((user) => user.name.trim());
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 321 - Find an array item

Goal: Implement the browser-side task: find an array item.

```
const match = users.find((user) => user.id === wantedId);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 322 - Check any array item

Goal: Implement the browser-side task: check any array item.

```
const hasErrors = fields.some((field) => field.error);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 323 - Check every array item

Goal: Implement the browser-side task: check every array item.

```
const complete = tasks.every((task) => task.done);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 324 - Sum values

Goal: Implement the browser-side task: sum values.

```
const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 325 - Sort without mutation

Goal: Implement the browser-side task: sort without mutation.

```
const sorted = users.toSorted((a, b) => a.name.localeCompare(b.name));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 326 - Replace array item immutably

Goal: Implement the browser-side task: replace array item immutably.

```
const next = items.with(2, { ...items[2], selected: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 327 - Structured clone

Goal: Implement the browser-side task: structured clone.

```
const independentCopy = structuredClone(settings);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 328 - Optional chaining

Goal: Implement the browser-side task: optional chaining.

```
const city = account?.profile?.address?.city ?? 'Unknown';
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 329 - Nullish default

Goal: Implement the browser-side task: nullish default.

```
const pageSize = options.pageSize ?? 25;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 330 - Object destructuring defaults

Goal: Implement the browser-side task: object destructuring defaults.

```
const { theme = 'system', density = 'comfortable' } = preferences;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 331 - Object rest

Goal: Implement the browser-side task: object rest.

```
const { password, ...safeUser } = user;  
console.log(safeUser);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 332 - Build object conditionally

Goal: Implement the browser-side task: build object conditionally.

```
const payload = { name, ...(email ? { email } : {}) };
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 333 - Convert object entries

Goal: Implement the browser-side task: convert object entries.

```
const query = Object.entries(filters).filter(([ , value]) => value != null);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 334 - URL construction

Goal: Implement the browser-side task: url construction.

```
const url = new URL('/search', location.origin);  
url.searchParams.set('q', 'css grid');  
console.log(url.href);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 335 - Parse a URL safely

Goal: Implement the browser-side task: parse a url safely.

```
function parseHttpRequest(value) {  
  try {  
    const url = new URL(value);  
    return ['http:', 'https:'].includes(url.protocol) ? url : null;  
  } catch { return null; }  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 336 - Generate a UUID

Goal: Implement the browser-side task: generate a uuid.

```
const id = crypto.randomUUID();  
console.log(id);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 337 - Generate random bytes

Goal: Implement the browser-side task: generate random bytes.

```
const bytes = crypto.getRandomValues(new Uint8Array(16));  
console.log(bytes);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 338 - Measure performance

Goal: Implement the browser-side task: measure performance.

```
performance.mark('render-start');  
render();  
performance.mark('render-end');  
performance.measure('render', 'render-start', 'render-end');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 339 - Read navigation timing

Goal: Implement the browser-side task: read navigation timing.

```
const [nav] = performance.getEntriesByType('navigation');  
console.log(nav?.domContentLoadedEventEnd);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 340 - Observe long tasks

Goal: Implement the browser-side task: observe long tasks.

```
const observer = new PerformanceObserver((list) => {  
  list.getEntries().forEach((entry) => console.log(entry.duration));  
});  
observer.observe({ type: 'longtask', buffered: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 341 - Queue a microtask

Goal: Implement the browser-side task: queue a microtask.

```
queueMicrotask(() => console.log('After current synchronous work'));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 342 - Use a Web Worker

Goal: Implement the browser-side task: use a web worker.

```
const worker = new Worker('/js/worker.js', { type: 'module' });
worker.postMessage({ values: [1, 2, 3] });
worker.onmessage = ({ data }) => console.log(data);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 343 - Register a service worker

Goal: Implement the browser-side task: register a service worker.

```
if ('serviceWorker' in navigator) {  
  await navigator.serviceWorker.register('/sw.js');  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 344 - Read cache storage

Goal: Implement the browser-side task: read cache storage.

```
const cache = await caches.open('assets-v1');  
const match = await cache.match('/styles.css');  
console.log(Boolean(match));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 345 - Canvas 2D setup

Goal: Implement the browser-side task: canvas 2d setup.

```
const canvas = document.querySelector('canvas');
const ctx = canvas?.getContext('2d');
if (ctx) { ctx.fillStyle = '#7c3aed'; ctx.fillRect(20, 20, 120, 80); }
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 346 - Respect device pixel ratio on canvas

Goal: Implement the browser-side task: respect device pixel ratio on canvas.

```
const canvas = document.querySelector('canvas');
const dpr = devicePixelRatio || 1;
if (canvas) {
  const { width, height } = canvas.getBoundingClientRect();
  canvas.width = Math.round(width * dpr);
  canvas.height = Math.round(height * dpr);
  canvas.getContext('2d').scale(dpr, dpr);
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 347 - Play an animation

Goal: Implement the browser-side task: play an animation.

```
const animation = document.querySelector('.card')?.animate(
  [{ opacity: 0, transform: 'translateY(12px)' }, { opacity: 1, transform: 'none' }],
  { duration: 240, easing: 'ease-out', fill: 'both' }
);
await animation?.finished;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 348 - Cancel Web Animation

Goal: Implement the browser-side task: cancel web animation.

```
const animation = document.querySelector('.spinner')?.getAnimations()[0];
animation?.cancel();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 349 - Pause CSS animations

Goal: Implement the browser-side task: pause css animations.

```
document.querySelectorAll('.animated').forEach((el) => {  
  el.getAnimations().forEach((animation) => animation.pause());  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 350 - Build a keyboard roving tabindex

Goal: Implement the browser-side task: build a keyboard roving tabindex.

```
const tabs = [...document.querySelectorAll('[role=tab]')];
tabs.forEach((tab, index) => {
  tab.tabIndex = index === 0 ? 0 : -1;
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 351 - Announce status accessibly

Goal: Implement the browser-side task: announce status accessibly.

```
const live = document.querySelector('[role=status]');  
if (live) live.textContent = '3 results loaded';
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 352 - Toggle aria-expanded

Goal: Implement the browser-side task: toggle aria-expanded.

```
const trigger = document.querySelector('[aria-controls=menu]);
trigger?.addEventListener('click', () => {
  const open = trigger.getAttribute('aria-expanded') === 'true';
  trigger.setAttribute('aria-expanded', String(!open));
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 353 - Escape closes a popover

Goal: Implement the browser-side task: escape closes a popover.

```
document.addEventListener('keydown', (event) => {  
  if (event.key === 'Escape') document.querySelector('[popover]:popover-open')?.hidePopover();  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 354 - Use native popover API

Goal: Implement the browser-side task: use native popover api.

```
const popover = document.querySelector('#help-popover');  
popover?.showPopover();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 355 - Inert background region

Goal: Implement the browser-side task: inert background region.

```
const main = document.querySelector('main');  
if (main) main.inert = true;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 356 - Restore interactive region

Goal: Implement the browser-side task: restore interactive region.

```
const main = document.querySelector('main');  
if (main) main.inert = false;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 357 - Copy current URL

Goal: Implement the browser-side task: copy current url.

```
await navigator.clipboard.writeText(location.href);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 358 - Detect coarse pointer

Goal: Implement the browser-side task: detect coarse pointer.

```
const touchLike = matchMedia('(pointer: coarse)').matches;  
console.log({ touchLike });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 359 - Feature detect an API

Goal: Implement the browser-side task: feature detect an api.

```
if ('IntersectionObserver' in window) {  
  console.log('IntersectionObserver supported');  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 360 - Dynamic module import

Goal: Implement the browser-side task: dynamic module import.

```
const { initGallery } = await import('./gallery.js');  
initGallery();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 361 - Import only after interaction

Goal: Implement the browser-side task: import only after interaction.

```
document.querySelector('.open-editor')?.addEventListener('click', async () => {  
  const { mountEditor } = await import('./editor.js');  
  mountEditor();  
}, { once: true });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 362 - Safe JSON response parsing

Goal: Implement the browser-side task: safe json response parsing.

```
async function readJson(response) {  
  const type = response.headers.get('content-type') ?? '';  
  if (!type.includes('application/json')) throw new TypeError('Expected JSON');  
  return response.json();  
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 363 - Abort previous search request

Goal: Implement the browser-side task: abort previous search request.

```
let controller;
async function search(q) {
  controller?.abort();
  controller = new AbortController();
  return fetch(`/api/search?q=${encodeURIComponent(q)}`, { signal: controller.signal });
}
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 364 - Exponential retry delay

Goal: Implement the browser-side task: exponential retry delay.

```
const retryDelay = (attempt) => Math.min(1000 * 2 ** attempt, 8000);  
console.log(retryDelay(3));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with `innerHTML`, and do not expose secrets in client-side JavaScript.

Snippet 365 - Sanitize by using textContent

Goal: Implement the browser-side task: sanitize by using textcontent.

```
const message = document.querySelector('.message');
const untrusted = '<img src=x onerror=alert(1)>';
if (message) message.textContent = untrusted;
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 366 - Avoid javascript URLs

Goal: Implement the browser-side task: avoid javascript urls.

```
const candidate = new URL(userProvidedUrl, location.origin);
if (!['http:', 'https:'].includes(candidate.protocol)) throw new Error('Unsupported URL
scheme');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 367 - Read CSRF token from metadata

Goal: Implement the browser-side task: read csrf token from metadata.

```
const csrf = document.querySelector('meta[name=csrf-token']?.content;  
console.log(Boolean(csrf));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 368 - Use same-origin credentials

Goal: Implement the browser-side task: use same-origin credentials.

```
const response = await fetch('/api/account', { credentials: 'same-origin' });  
if (!response.ok) throw new Error('Request failed');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 369 - Read selected option text

Goal: Implement the browser-side task: read selected option text.

```
const select = document.querySelector('#theme');
const label = select?.selectedOptions[0]?.textContent ?? '';
console.log(label);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 370 - Move focus to first invalid field

Goal: Implement the browser-side task: move focus to first invalid field.

```
const form = document.querySelector('form');
const invalid = form?.querySelector(':invalid');
invalid?.focus();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 371 - Preserve scroll position in history

Goal: Implement the browser-side task: preserve scroll position in history.

```
history.scrollRestoration = 'manual';  
sessionStorage.setItem('scrollY', String(scrollY));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 372 - Detect document visibility

Goal: Implement the browser-side task: detect document visibility.

```
document.addEventListener('visibilitychange', () => {  
  console.log(document.visibilityState);  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 373 - Pause work in hidden tabs

Goal: Implement the browser-side task: pause work in hidden tabs.

```
document.addEventListener('visibilitychange', () => {  
  if (document.hidden) stopPolling();  
  else startPolling();  
});
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 374 - Read network connection hint

Goal: Implement the browser-side task: read network connection hint.

```
const connection = navigator.connection;
if (connection) console.log(connection.effectiveType, connection.saveData);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 375 - Detect Save-Data preference

Goal: Implement the browser-side task: detect save-data preference.

```
const saveData = navigator.connection?.saveData === true;
console.log({ saveData });
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 376 - Request geolocation once

Goal: Implement the browser-side task: request geolocation once.

```
navigator.geolocation?.getCurrentPosition(
  ({ coords }) => console.log(coords.latitude, coords.longitude),
  (error) => console.warn(error.message),
  { enableHighAccuracy: false, timeout: 5000 }
);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 377 - Create a media query listener

Goal: Implement the browser-side task: create a media query listener.

```
const wide = matchMedia('(min-width: 60rem)');
wide.addEventListener('change', ({ matches }) => console.log({ wide: matches }));
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 378 - Toggle a boolean data state

Goal: Implement the browser-side task: toggle a boolean data state.

```
const panel = document.querySelector('.panel');  
if (panel) panel.dataset.open = String(panel.dataset.open !== 'true');
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 379 - Read hash target safely

Goal: Implement the browser-side task: read hash target safely.

```
const id = decodeURIComponent(location.hash.slice(1));
const target = id ? document.getElementById(id) : null;
target?.scrollIntoView();
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

Snippet 380 - Wait for fonts before measuring

Goal: Implement the browser-side task: wait for fonts before measuring.

```
await document.fonts.ready;
const width = document.querySelector('.headline')?.getBoundingClientRect().width;
console.log(width);
```

Step by step

1. Put the code in a module script or a JavaScript file loaded after the relevant markup exists.
2. Replace example selectors, endpoints, IDs, and data with values from your page.
3. Handle the empty/error state before adding animation or visual polish.
4. Exercise the action with mouse, keyboard, touch when relevant, and inspect the console/network panel for failures.

Verify: Trigger both the normal path and at least one failure/empty path; verify there are no uncaught exceptions and the UI state stays understandable.

Gotcha: Browser APIs differ in permissions and support. Feature-detect optional APIs, do not inject untrusted strings with innerHTML, and do not expose secrets in client-side JavaScript.

UI mini-projects

Recipes 381-430

Snippet 381 - Screenshot-inspired 3D photo gallery

Goal: Use perspective, horizontal cards, and keyboard buttons without copying the pictured creator's code.

```
<!-- HTML -->
<section class="demo screenshot-inspired-3d-photo-gallery" aria-labelledby="screenshot-
inspired-3d-photo-gallery-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 01</p>
    <h2 id="screenshot-inspired-3d-photo-gallery-title">Screenshot-inspired 3D photo
    gallery</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.screenshot-inspired-3d-photo-gallery {
  --accent: hsl(47 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.screenshot-inspired-3d-photo-gallery .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.screenshot-inspired-3d-photo-gallery .demo__action { min-block-size: 44px; padding-inline:
1rem; }
@media (prefers-reduced-motion: no-preference) {
  .screenshot-inspired-3d-photo-gallery .demo__content { transition: transform 180ms ease; }
  .screenshot-inspired-3d-photo-gallery:has(.demo__action[aria-pressed="true"]) .demo__content
  { transform: translateY(-4px); }
}

// JavaScript
const root1 = document.querySelector('.screenshot-inspired-3d-photo-gallery');
const button1 = root1?.querySelector('.demo__action');
const status1 = root1?.querySelector('.demo__status');
button1?.addEventListener('click', () => {
  const next = button1.getAttribute('aria-pressed') !== 'true';
  button1.setAttribute('aria-pressed', String(next));
  if (status1) status1.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 382 - CSS cheat-sheet grid

Goal: Lay out reference cards that collapse cleanly on small screens.

```
<!-- HTML -->
<section class="demo css-cheat-sheet-grid" aria-labelledby="css-cheat-sheet-grid-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 02</p>
    <h2 id="css-cheat-sheet-grid-title">CSS cheat-sheet grid</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.css-cheat-sheet-grid {
  --accent: hsl(94 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.css-cheat-sheet-grid .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.css-cheat-sheet-grid .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .css-cheat-sheet-grid .demo__content { transition: transform 180ms ease; }
  .css-cheat-sheet-grid:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root2 = document.querySelector('.css-cheat-sheet-grid');
const button2 = root2?.querySelector('.demo__action');
const status2 = root2?.querySelector('.demo__status');
button2?.addEventListener('click', () => {
  const next = button2.getAttribute('aria-pressed') !== 'true';
  button2.setAttribute('aria-pressed', String(next));
  if (status2) status2.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 383 - Animated vehicle hero

Goal: Build a cinematic product hero with layered copy and a transform-based media reveal.

```
<!-- HTML -->
<section class="demo animated-vehicle-hero" aria-labelledby="animated-vehicle-hero-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 03</p>
    <h2 id="animated-vehicle-hero-title">Animated vehicle hero</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.animated-vehicle-hero {
  --accent: hsl(141 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.animated-vehicle-hero .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.animated-vehicle-hero .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .animated-vehicle-hero .demo__content { transition: transform 180ms ease; }
  .animated-vehicle-hero:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root3 = document.querySelector('.animated-vehicle-hero');
const button3 = root3?.querySelector('.demo__action');
const status3 = root3?.querySelector('.demo__status');
button3?.addEventListener('click', () => {
  const next = button3.getAttribute('aria-pressed') !== 'true';
  button3.setAttribute('aria-pressed', String(next));
  if (status3) status3.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 384 - Neon launch gallery

Goal: Create an accordion-like gallery with keyboard-friendly next/previous controls.

```
<!-- HTML -->
<section class="demo neon-launch-gallery" aria-labelledby="neon-launch-gallery-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 04</p>
    <h2 id="neon-launch-gallery-title">Neon launch gallery</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.neon-launch-gallery {
  --accent: hsl(188 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.neon-launch-gallery .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.neon-launch-gallery .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .neon-launch-gallery .demo__content { transition: transform 180ms ease; }
  .neon-launch-gallery:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root4 = document.querySelector('.neon-launch-gallery');
const button4 = root4?.querySelector('.demo__action');
const status4 = root4?.querySelector('.demo__status');
button4?.addEventListener('click', () => {
  const next = button4.getAttribute('aria-pressed') !== 'true';
  button4.setAttribute('aria-pressed', String(next));
  if (status4) status4.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 385 - Animated sidebar

Goal: Build an icon rail that expands labels while preserving focus and touch targets.

```
<!-- HTML -->
<section class="demo animated-sidebar" aria-labelledby="animated-sidebar-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 05</p>
    <h2 id="animated-sidebar-title">Animated sidebar</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.animated-sidebar {
  --accent: hsl(235 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.animated-sidebar .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.animated-sidebar .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .animated-sidebar .demo__content { transition: transform 180ms ease; }
  .animated-sidebar:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root5 = document.querySelector('.animated-sidebar');
const button5 = root5?.querySelector('.demo__action');
const status5 = root5?.querySelector('.demo__status');
button5?.addEventListener('click', () => {
  const next = button5.getAttribute('aria-pressed') !== 'true';
  button5.setAttribute('aria-pressed', String(next));
  if (status5) status5.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 386 - Purple login panel

Goal: Create a responsive split login card with a decorative gradient panel.

```
<!-- HTML -->
<section class="demo purple-login-panel" aria-labelledby="purple-login-panel-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 06</p>
    <h2 id="purple-login-panel-title">Purple login panel</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.purple-login-panel {
  --accent: hsl(282 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.purple-login-panel .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.purple-login-panel .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .purple-login-panel .demo__content { transition: transform 180ms ease; }
  .purple-login-panel:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root6 = document.querySelector('.purple-login-panel');
const button6 = root6?.querySelector('.demo__action');
const status6 = root6?.querySelector('.demo__status');
button6?.addEventListener('click', () => {
  const next = button6.getAttribute('aria-pressed') !== 'true';
  button6.setAttribute('aria-pressed', String(next));
  if (status6) status6.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 387 - Playful registration form

Goal: Build a bright form composition with progressive form sections.

```
<!-- HTML -->
<section class="demo playful-registration-form" aria-labelledby="playful-registration-form-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 07</p>
    <h2 id="playful-registration-form-title">Playful registration form</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.playful-registration-form {
  --accent: hsl(329 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.playful-registration-form .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.playful-registration-form .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .playful-registration-form .demo__content { transition: transform 180ms ease; }
  .playful-registration-form:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root7 = document.querySelector('.playful-registration-form');
const button7 = root7?.querySelector('.demo__action');
const status7 = root7?.querySelector('.demo__status');
button7?.addEventListener('click', () => {
  const next = button7.getAttribute('aria-pressed') !== 'true';
  button7.setAttribute('aria-pressed', String(next));
  if (status7) status7.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 388 - Animated 404 page

Goal: Create a useful not-found state with restrained decorative motion.

```
<!-- HTML -->
<section class="demo animated-404-page" aria-labelledby="animated-404-page-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 08</p>
    <h2 id="animated-404-page-title">Animated 404 page</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.animated-404-page {
  --accent: hsl(16 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.animated-404-page .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.animated-404-page .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .animated-404-page .demo__content { transition: transform 180ms ease; }
  .animated-404-page:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root8 = document.querySelector('.animated-404-page');
const button8 = root8?.querySelector('.demo__action');
const status8 = root8?.querySelector('.demo__status');
button8?.addEventListener('click', () => {
  const next = button8.getAttribute('aria-pressed') !== 'true';
  button8.setAttribute('aria-pressed', String(next));
  if (status8) status8.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 389 - Modern team carousel

Goal: Build a center-emphasized carousel with buttons and visible names.

```
<!-- HTML -->
<section class="demo modern-team-carousel" aria-labelledby="modern-team-carousel-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 09</p>
    <h2 id="modern-team-carousel-title">Modern team carousel</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.modern-team-carousel {
  --accent: hsl(63 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.modern-team-carousel .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.modern-team-carousel .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .modern-team-carousel .demo__content { transition: transform 180ms ease; }
  .modern-team-carousel:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root9 = document.querySelector('.modern-team-carousel');
const button9 = root9?.querySelector('.demo__action');
const status9 = root9?.querySelector('.demo__status');
button9?.addEventListener('click', () => {
  const next = button9.getAttribute('aria-pressed') !== 'true';
  button9.setAttribute('aria-pressed', String(next));
  if (status9) status9.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 390 - Three-CSS-tools demo board

Goal: Show a gradient, grid, and keyframe effect together as a learning sandbox.

```
<!-- HTML -->
<section class="demo three-css-tools-demo-board" aria-labelledby="three-css-tools-demo-board-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 10</p>
    <h2 id="three-css-tools-demo-board-title">Three-CSS-tools demo board</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.three-css-tools-demo-board {
  --accent: hsl(110 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.three-css-tools-demo-board .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.three-css-tools-demo-board .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .three-css-tools-demo-board .demo__content { transition: transform 180ms ease; }
  .three-css-tools-demo-board:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root10 = document.querySelector('.three-css-tools-demo-board');
const button10 = root10?.querySelector('.demo__action');
const status10 = root10?.querySelector('.demo__status');
button10?.addEventListener('click', () => {
  const next = button10.getAttribute('aria-pressed') !== 'true';
  button10.setAttribute('aria-pressed', String(next));
  if (status10) status10.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 391 - Responsive navigation bar

Goal: Build a compact header that wraps cleanly and exposes a menu control.

```
<!-- HTML -->
<section class="demo responsive-navigation-bar" aria-labelledby="responsive-navigation-bar-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 11</p>
    <h2 id="responsive-navigation-bar-title">Responsive navigation bar</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.responsive-navigation-bar {
  --accent: hsl(157 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.responsive-navigation-bar .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.responsive-navigation-bar .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .responsive-navigation-bar .demo__content { transition: transform 180ms ease; }
  .responsive-navigation-bar:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root11 = document.querySelector('.responsive-navigation-bar');
const button11 = root11?.querySelector('.demo__action');
const status11 = root11?.querySelector('.demo__status');
button11?.addEventListener('click', () => {
  const next = button11.getAttribute('aria-pressed') !== 'true';
  button11.setAttribute('aria-pressed', String(next));
  if (status11) status11.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 392 - Hero with call-to-action

Goal: Create a two-column hero with a fluid heading and action row.

```
<!-- HTML -->
<section class="demo hero-with-call-to-action" aria-labelledby="hero-with-call-to-action-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 12</p>
    <h2 id="hero-with-call-to-action-title">Hero with call-to-action</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.hero-with-call-to-action {
  --accent: hsl(204 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.hero-with-call-to-action .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.hero-with-call-to-action .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .hero-with-call-to-action .demo__content { transition: transform 180ms ease; }
  .hero-with-call-to-action:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root12 = document.querySelector('.hero-with-call-to-action');
const button12 = root12?.querySelector('.demo__action');
const status12 = root12?.querySelector('.demo__status');
button12?.addEventListener('click', () => {
  const next = button12.getAttribute('aria-pressed') !== 'true';
  button12.setAttribute('aria-pressed', String(next));
  if (status12) status12.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 393 - Feature card grid

Goal: Use auto-fit Grid so cards respond without breakpoint micromanagement.

```
<!-- HTML -->
<section class="demo feature-card-grid" aria-labelledby="feature-card-grid-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 13</p>
    <h2 id="feature-card-grid-title">Feature card grid</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.feature-card-grid {
  --accent: hsl(251 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.feature-card-grid .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.feature-card-grid .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .feature-card-grid .demo__content { transition: transform 180ms ease; }
  .feature-card-grid:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root13 = document.querySelector('.feature-card-grid');
const button13 = root13?.querySelector('.demo__action');
const status13 = root13?.querySelector('.demo__status');
button13?.addEventListener('click', () => {
  const next = button13.getAttribute('aria-pressed') !== 'true';
  button13.setAttribute('aria-pressed', String(next));
  if (status13) status13.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 394 - Pricing cards

Goal: Build comparable plans with semantic headings and a highlighted recommendation.

```
<!-- HTML -->
<section class="demo pricing-cards" aria-labelledby="pricing-cards-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 14</p>
    <h2 id="pricing-cards-title">Pricing cards</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.pricing-cards {
  --accent: hsl(298 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.pricing-cards .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.pricing-cards .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .pricing-cards .demo__content { transition: transform 180ms ease; }
  .pricing-cards:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root14 = document.querySelector('.pricing-cards');
const button14 = root14?.querySelector('.demo__action');
const status14 = root14?.querySelector('.demo__status');
button14?.addEventListener('click', () => {
  const next = button14.getAttribute('aria-pressed') !== 'true';
  button14.setAttribute('aria-pressed', String(next));
  if (status14) status14.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 395 - FAQ accordion

Goal: Use native details/summary for keyboard-operable disclosures.

```
<!-- HTML -->
<section class="demo faq-accordion" aria-labelledby="faq-accordion-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 15</p>
    <h2 id="faq-accordion-title">FAQ accordion</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.faq-accordion {
  --accent: hsl(345 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.faq-accordion .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.faq-accordion .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .faq-accordion .demo__content { transition: transform 180ms ease; }
  .faq-accordion:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root15 = document.querySelector('.faq-accordion');
const button15 = root15?.querySelector('.demo__action');
const status15 = root15?.querySelector('.demo__status');
button15?.addEventListener('click', () => {
  const next = button15.getAttribute('aria-pressed') !== 'true';
  button15.setAttribute('aria-pressed', String(next));
  if (status15) status15.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 396 - Tabbed interface skeleton

Goal: Build an ARIA tab scaffold with explicit relationships.

```
<!-- HTML -->
<section class="demo tabbed-interface-skeleton" aria-labelledby="tabbed-interface-skeleton-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 16</p>
    <h2 id="tabbed-interface-skeleton-title">Tabbed interface skeleton</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.tabbed-interface-skeleton {
  --accent: hsl(32 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.tabbed-interface-skeleton .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.tabbed-interface-skeleton .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .tabbed-interface-skeleton .demo__content { transition: transform 180ms ease; }
  .tabbed-interface-skeleton:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root16 = document.querySelector('.tabbed-interface-skeleton');
const button16 = root16?.querySelector('.demo__action');
const status16 = root16?.querySelector('.demo__status');
button16?.addEventListener('click', () => {
  const next = button16.getAttribute('aria-pressed') !== 'true';
  button16.setAttribute('aria-pressed', String(next));
  if (status16) status16.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 397 - Native modal

Goal: Use dialog for focus management and modal semantics.

```
<!-- HTML -->
<section class="demo native-modal" aria-labelledby="native-modal-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 17</p>
    <h2 id="native-modal-title">Native modal</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.native-modal {
  --accent: hsl(79 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.native-modal .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.native-modal .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .native-modal .demo__content { transition: transform 180ms ease; }
  .native-modal:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root17 = document.querySelector('.native-modal');
const button17 = root17?.querySelector('.demo__action');
const status17 = root17?.querySelector('.demo__status');
button17?.addEventListener('click', () => {
  const next = button17.getAttribute('aria-pressed') !== 'true';
  button17.setAttribute('aria-pressed', String(next));
  if (status17) status17.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 398 - Off-canvas drawer

Goal: Build a side drawer with a visible close control and inert background strategy.

```
<!-- HTML -->
<section class="demo off-canvas-drawer" aria-labelledby="off-canvas-drawer-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 18</p>
    <h2 id="off-canvas-drawer-title">Off-canvas drawer</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.off-canvas-drawer {
  --accent: hsl(126 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.off-canvas-drawer .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.off-canvas-drawer .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .off-canvas-drawer .demo__content { transition: transform 180ms ease; }
  .off-canvas-drawer:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root18 = document.querySelector('.off-canvas-drawer');
const button18 = root18?.querySelector('.demo__action');
const status18 = root18?.querySelector('.demo__status');
button18?.addEventListener('click', () => {
  const next = button18.getAttribute('aria-pressed') !== 'true';
  button18.setAttribute('aria-pressed', String(next));
  if (status18) status18.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 399 - Toast status message

Goal: Announce non-blocking status changes without stealing focus.

```
<!-- HTML -->
<section class="demo toast-status-message" aria-labelledby="toast-status-message-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 19</p>
    <h2 id="toast-status-message-title">Toast status message</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.toast-status-message {
  --accent: hsl(173 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.toast-status-message .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.toast-status-message .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .toast-status-message .demo__content { transition: transform 180ms ease; }
  .toast-status-message:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root19 = document.querySelector('.toast-status-message');
const button19 = root19?.querySelector('.demo__action');
const status19 = root19?.querySelector('.demo__status');
button19?.addEventListener('click', () => {
  const next = button19.getAttribute('aria-pressed') !== 'true';
  button19.setAttribute('aria-pressed', String(next));
  if (status19) status19.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 400 - Tooltip with popover

Goal: Build a small help surface using the native popover API.

```
<!-- HTML -->
<section class="demo tooltip-with-popover" aria-labelledby="tooltip-with-popover-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 20</p>
    <h2 id="tooltip-with-popover-title">Tooltip with popover</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.tooltip-with-popover {
  --accent: hsl(220 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.tooltip-with-popover .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.tooltip-with-popover .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .tooltip-with-popover .demo__content { transition: transform 180ms ease; }
  .tooltip-with-popover:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root20 = document.querySelector('.tooltip-with-popover');
const button20 = root20?.querySelector('.demo__action');
const status20 = root20?.querySelector('.demo__status');
button20?.addEventListener('click', () => {
  const next = button20.getAttribute('aria-pressed') !== 'true';
  button20.setAttribute('aria-pressed', String(next));
  if (status20) status20.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 401 - Breadcrumb trail

Goal: Create semantic breadcrumb navigation with an accessible label.

```
<!-- HTML -->
<section class="demo breadcrumb-trail" aria-labelledby="breadcrumb-trail-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 21</p>
    <h2 id="breadcrumb-trail-title">Breadcrumb trail</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.breadcrumb-trail {
  --accent: hsl(267 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.breadcrumb-trail .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.breadcrumb-trail .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .breadcrumb-trail .demo__content { transition: transform 180ms ease; }
  .breadcrumb-trail:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root21 = document.querySelector('.breadcrumb-trail');
const button21 = root21?.querySelector('.demo__action');
const status21 = root21?.querySelector('.demo__status');
button21?.addEventListener('click', () => {
  const next = button21.getAttribute('aria-pressed') !== 'true';
  button21.setAttribute('aria-pressed', String(next));
  if (status21) status21.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 402 - Pagination bar

Goal: Provide real links and indicate the current page.

```
<!-- HTML -->
<section class="demo pagination-bar" aria-labelledby="pagination-bar-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 22</p>
    <h2 id="pagination-bar-title">Pagination bar</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.pagination-bar {
  --accent: hsl(314 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.pagination-bar .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.pagination-bar .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .pagination-bar .demo__content { transition: transform 180ms ease; }
  .pagination-bar:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root22 = document.querySelector('.pagination-bar');
const button22 = root22?.querySelector('.demo__action');
const status22 = root22?.querySelector('.demo__status');
button22?.addEventListener('click', () => {
  const next = button22.getAttribute('aria-pressed') !== 'true';
  button22.setAttribute('aria-pressed', String(next));
  if (status22) status22.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 403 - Search results list

Goal: Combine a search form, count live region, and results list.

```
<!-- HTML -->
<section class="demo search-results-list" aria-labelledby="search-results-list-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 23</p>
    <h2 id="search-results-list-title">Search results list</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.search-results-list {
  --accent: hsl(1 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.search-results-list .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.search-results-list .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .search-results-list .demo__content { transition: transform 180ms ease; }
  .search-results-list:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root23 = document.querySelector('.search-results-list');
const button23 = root23?.querySelector('.demo__action');
const status23 = root23?.querySelector('.demo__status');
button23?.addEventListener('click', () => {
  const next = button23.getAttribute('aria-pressed') !== 'true';
  button23.setAttribute('aria-pressed', String(next));
  if (status23) status23.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 404 - Filter chips

Goal: Create toggle buttons that expose pressed state.

```
<!-- HTML -->
<section class="demo filter-chips" aria-labelledby="filter-chips-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 24</p>
    <h2 id="filter-chips-title">Filter chips</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.filter-chips {
  --accent: hsl(48 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.filter-chips .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.filter-chips .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .filter-chips .demo__content { transition: transform 180ms ease; }
  .filter-chips:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root24 = document.querySelector('.filter-chips');
const button24 = root24?.querySelector('.demo__action');
const status24 = root24?.querySelector('.demo__status');
button24?.addEventListener('click', () => {
  const next = button24.getAttribute('aria-pressed') !== 'true';
  button24.setAttribute('aria-pressed', String(next));
  if (status24) status24.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 405 - Profile card

Goal: Combine responsive image sizing, semantic content, and action links.

```
<!-- HTML -->
<section class="demo profile-card" aria-labelledby="profile-card-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 25</p>
    <h2 id="profile-card-title">Profile card</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.profile-card {
  --accent: hsl(95 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.profile-card .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.profile-card .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .profile-card .demo__content { transition: transform 180ms ease; }
  .profile-card:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root25 = document.querySelector('.profile-card');
const button25 = root25?.querySelector('.demo__action');
const status25 = root25?.querySelector('.demo__status');
button25?.addEventListener('click', () => {
  const next = button25.getAttribute('aria-pressed') !== 'true';
  button25.setAttribute('aria-pressed', String(next));
  if (status25) status25.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 406 - Stat cards

Goal: Show key metrics with tabular numbers and fluid wrapping.

```
<!-- HTML -->
<section class="demo stat-cards" aria-labelledby="stat-cards-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 26</p>
    <h2 id="stat-cards-title">Stat cards</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.stat-cards {
  --accent: hsl(142 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.stat-cards .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.stat-cards .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .stat-cards .demo__content { transition: transform 180ms ease; }
  .stat-cards:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root26 = document.querySelector('.stat-cards');
const button26 = root26?.querySelector('.demo__action');
const status26 = root26?.querySelector('.demo__status');
button26?.addEventListener('click', () => {
  const next = button26.getAttribute('aria-pressed') !== 'true';
  button26.setAttribute('aria-pressed', String(next));
  if (status26) status26.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 407 - Timeline

Goal: Render chronological events with an ordered list and CSS marker line.

```
<!-- HTML -->
<section class="demo timeline" aria-labelledby="timeline-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 27</p>
    <h2 id="timeline-title">Timeline</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.timeline {
  --accent: hsl(189 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.timeline .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.timeline .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .timeline .demo__content { transition: transform 180ms ease; }
  .timeline:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root27 = document.querySelector('.timeline');
const button27 = root27?.querySelector('.demo__action');
const status27 = root27?.querySelector('.demo__status');
button27?.addEventListener('click', () => {
  const next = button27.getAttribute('aria-pressed') !== 'true';
  button27.setAttribute('aria-pressed', String(next));
  if (status27) status27.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 408 - Progress steps

Goal: Model a multi-step flow and identify the current step.

```
<!-- HTML -->
<section class="demo progress-steps" aria-labelledby="progress-steps-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 28</p>
    <h2 id="progress-steps-title">Progress steps</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.progress-steps {
  --accent: hsl(236 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.progress-steps .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.progress-steps .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .progress-steps .demo__content { transition: transform 180ms ease; }
  .progress-steps:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root28 = document.querySelector('.progress-steps');
const button28 = root28?.querySelector('.demo__action');
const status28 = root28?.querySelector('.demo__status');
button28?.addEventListener('click', () => {
  const next = button28.getAttribute('aria-pressed') !== 'true';
  button28.setAttribute('aria-pressed', String(next));
  if (status28) status28.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 409 - Skeleton loader

Goal: Build a content placeholder that disables animation for reduced-motion users.

```
<!-- HTML -->
<section class="demo skeleton-loader" aria-labelledby="skeleton-loader-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 29</p>
    <h2 id="skeleton-loader-title">Skeleton loader</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.skeleton-loader {
  --accent: hsl(283 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.skeleton-loader .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.skeleton-loader .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .skeleton-loader .demo__content { transition: transform 180ms ease; }
  .skeleton-loader:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root29 = document.querySelector('.skeleton-loader');
const button29 = root29?.querySelector('.demo__action');
const status29 = root29?.querySelector('.demo__status');
button29?.addEventListener('click', () => {
  const next = button29.getAttribute('aria-pressed') !== 'true';
  button29.setAttribute('aria-pressed', String(next));
  if (status29) status29.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 410 - Empty state

Goal: Provide explanation and a primary recovery action rather than decoration alone.

```
<!-- HTML -->
<section class="demo empty-state" aria-labelledby="empty-state-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 30</p>
    <h2 id="empty-state-title">Empty state</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.empty-state {
  --accent: hsl(330 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.empty-state .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.empty-state .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .empty-state .demo__content { transition: transform 180ms ease; }
  .empty-state:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root30 = document.querySelector('.empty-state');
const button30 = root30?.querySelector('.demo__action');
const status30 = root30?.querySelector('.demo__status');
button30?.addEventListener('click', () => {
  const next = button30.getAttribute('aria-pressed') !== 'true';
  button30.setAttribute('aria-pressed', String(next));
  if (status30) status30.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 411 - Error alert

Goal: Build a focusable error summary that links to invalid fields.

```
<!-- HTML -->
<section class="demo error-alert" aria-labelledby="error-alert-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 31</p>
    <h2 id="error-alert-title">Error alert</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.error-alert {
  --accent: hsl(17 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.error-alert .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.error-alert .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .error-alert .demo__content { transition: transform 180ms ease; }
  .error-alert:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root31 = document.querySelector('.error-alert');
const button31 = root31?.querySelector('.demo__action');
const status31 = root31?.querySelector('.demo__status');
button31?.addEventListener('click', () => {
  const next = button31.getAttribute('aria-pressed') !== 'true';
  button31.setAttribute('aria-pressed', String(next));
  if (status31) status31.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 412 - Notification center item

Goal: Group notification text, time, and dismiss control.

```
<!-- HTML -->
<section class="demo notification-center-item" aria-labelledby="notification-center-item-
  title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 32</p>
    <h2 id="notification-center-item-title">Notification center item</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.notification-center-item {
  --accent: hsl(64 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.notification-center-item .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.notification-center-item .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .notification-center-item .demo__content { transition: transform 180ms ease; }
  .notification-center-item:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root32 = document.querySelector('.notification-center-item');
const button32 = root32?.querySelector('.demo__action');
const status32 = root32?.querySelector('.demo__status');
button32?.addEventListener('click', () => {
  const next = button32.getAttribute('aria-pressed') !== 'true';
  button32.setAttribute('aria-pressed', String(next));
  if (status32) status32.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 413 - Command palette shell

Goal: Build a dialog-based searchable command list.

```
<!-- HTML -->
<section class="demo command-palette-shell" aria-labelledby="command-palette-shell-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 33</p>
    <h2 id="command-palette-shell-title">Command palette shell</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.command-palette-shell {
  --accent: hsl(111 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.command-palette-shell .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.command-palette-shell .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .command-palette-shell .demo__content { transition: transform 180ms ease; }
  .command-palette-shell:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root33 = document.querySelector('.command-palette-shell');
const button33 = root33?.querySelector('.demo__action');
const status33 = root33?.querySelector('.demo__status');
button33?.addEventListener('click', () => {
  const next = button33.getAttribute('aria-pressed') !== 'true';
  button33.setAttribute('aria-pressed', String(next));
  if (status33) status33.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 414 - Dropdown action menu

Goal: Use a button and popover surface rather than hover-only interaction.

```
<!-- HTML -->
<section class="demo dropdown-action-menu" aria-labelledby="dropdown-action-menu-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 34</p>
    <h2 id="dropdown-action-menu-title">Dropdown action menu</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.dropdown-action-menu {
  --accent: hsl(158 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.dropdown-action-menu .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.dropdown-action-menu .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .dropdown-action-menu .demo__content { transition: transform 180ms ease; }
  .dropdown-action-menu:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root34 = document.querySelector('.dropdown-action-menu');
const button34 = root34?.querySelector('.demo__action');
const status34 = root34?.querySelector('.demo__status');
button34?.addEventListener('click', () => {
  const next = button34.getAttribute('aria-pressed') !== 'true';
  button34.setAttribute('aria-pressed', String(next));
  if (status34) status34.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 415 - Responsive data table

Goal: Provide a horizontally scrollable table with caption and row/column headers.

```
<!-- HTML -->
<section class="demo responsive-data-table" aria-labelledby="responsive-data-table-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 35</p>
    <h2 id="responsive-data-table-title">Responsive data table</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.responsive-data-table {
  --accent: hsl(205 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.responsive-data-table .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.responsive-data-table .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .responsive-data-table .demo__content { transition: transform 180ms ease; }
  .responsive-data-table:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root35 = document.querySelector('.responsive-data-table');
const button35 = root35?.querySelector('.demo__action');
const status35 = root35?.querySelector('.demo__status');
button35?.addEventListener('click', () => {
  const next = button35.getAttribute('aria-pressed') !== 'true';
  button35.setAttribute('aria-pressed', String(next));
  if (status35) status35.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 416 - Comparison table

Goal: Keep plan labels as row headers so data stays understandable to assistive tech.

```
<!-- HTML -->
<section class="demo comparison-table" aria-labelledby="comparison-table-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 36</p>
    <h2 id="comparison-table-title">Comparison table</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.comparison-table {
  --accent: hsl(252 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.comparison-table .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.comparison-table .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .comparison-table .demo__content { transition: transform 180ms ease; }
  .comparison-table:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root36 = document.querySelector('.comparison-table');
const button36 = root36?.querySelector('.demo__action');
const status36 = root36?.querySelector('.demo__status');
button36?.addEventListener('click', () => {
  const next = button36.getAttribute('aria-pressed') !== 'true';
  button36.setAttribute('aria-pressed', String(next));
  if (status36) status36.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 417 - Contact form

Goal: Use native labels, autocomplete, required state, and a status region.

```
<!-- HTML -->
<section class="demo contact-form" aria-labelledby="contact-form-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 37</p>
    <h2 id="contact-form-title">Contact form</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.contact-form {
  --accent: hsl(299 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.contact-form .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.contact-form .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .contact-form .demo__content { transition: transform 180ms ease; }
  .contact-form:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root37 = document.querySelector('.contact-form');
const button37 = root37?.querySelector('.demo__action');
const status37 = root37?.querySelector('.demo__status');
button37?.addEventListener('click', () => {
  const next = button37.getAttribute('aria-pressed') !== 'true';
  button37.setAttribute('aria-pressed', String(next));
  if (status37) status37.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 418 - Newsletter form

Goal: Create a small email form with clear purpose and submission state.

```
<!-- HTML -->
<section class="demo newsletter-form" aria-labelledby="newsletter-form-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 38</p>
    <h2 id="newsletter-form-title">Newsletter form</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.newsletter-form {
  --accent: hsl(346 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.newsletter-form .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.newsletter-form .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .newsletter-form .demo__content { transition: transform 180ms ease; }
  .newsletter-form:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root38 = document.querySelector('.newsletter-form');
const button38 = root38?.querySelector('.demo__action');
const status38 = root38?.querySelector('.demo__status');
button38?.addEventListener('click', () => {
  const next = button38.getAttribute('aria-pressed') !== 'true';
  button38.setAttribute('aria-pressed', String(next));
  if (status38) status38.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 419 - Image lightbox

Goal: Use a dialog for a larger image preview with a meaningful alt text.

```
<!-- HTML -->
<section class="demo image-lightbox" aria-labelledby="image-lightbox-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 39</p>
    <h2 id="image-lightbox-title">Image lightbox</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.image-lightbox {
  --accent: hsl(33 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.image-lightbox .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.image-lightbox .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .image-lightbox .demo__content { transition: transform 180ms ease; }
  .image-lightbox:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root39 = document.querySelector('.image-lightbox');
const button39 = root39?.querySelector('.demo__action');
const status39 = root39?.querySelector('.demo__status');
button39?.addEventListener('click', () => {
  const next = button39.getAttribute('aria-pressed') !== 'true';
  button39.setAttribute('aria-pressed', String(next));
  if (status39) status39.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 420 - Masonry-like columns

Goal: Create a visual collage with CSS columns while noting source-order tradeoffs.

```
<!-- HTML -->
<section class="demo masonry-like-columns" aria-labelledby="masonry-like-columns-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 40</p>
    <h2 id="masonry-like-columns-title">Masonry-like columns</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.masonry-like-columns {
  --accent: hsl(80 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.masonry-like-columns .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.masonry-like-columns .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .masonry-like-columns .demo__content { transition: transform 180ms ease; }
  .masonry-like-columns:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root40 = document.querySelector('.masonry-like-columns');
const button40 = root40?.querySelector('.demo__action');
const status40 = root40?.querySelector('.demo__status');
button40?.addEventListener('click', () => {
  const next = button40.getAttribute('aria-pressed') !== 'true';
  button40.setAttribute('aria-pressed', String(next));
  if (status40) status40.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 421 - Horizontal scroll cards

Goal: Build a touch-friendly card rail with scroll snap.

```
<!-- HTML -->
<section class="demo horizontal-scroll-cards" aria-labelledby="horizontal-scroll-cards-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 41</p>
    <h2 id="horizontal-scroll-cards-title">Horizontal scroll cards</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.horizontal-scroll-cards {
  --accent: hsl(127 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.horizontal-scroll-cards .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.horizontal-scroll-cards .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .horizontal-scroll-cards .demo__content { transition: transform 180ms ease; }
  .horizontal-scroll-cards:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root41 = document.querySelector('.horizontal-scroll-cards');
const button41 = root41?.querySelector('.demo__action');
const status41 = root41?.querySelector('.demo__status');
button41?.addEventListener('click', () => {
  const next = button41.getAttribute('aria-pressed') !== 'true';
  button41.setAttribute('aria-pressed', String(next));
  if (status41) status41.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 422 - Before/after comparison

Goal: Use a range input to expose more or less of an overlay image.

```
<!-- HTML -->
<section class="demo before-after-comparison" aria-labelledby="before-after-comparison-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 42</p>
    <h2 id="before-after-comparison-title">Before/after comparison</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.before-after-comparison {
  --accent: hsl(174 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.before-after-comparison .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.before-after-comparison .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .before-after-comparison .demo__content { transition: transform 180ms ease; }
  .before-after-comparison:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root42 = document.querySelector('.before-after-comparison');
const button42 = root42?.querySelector('.demo__action');
const status42 = root42?.querySelector('.demo__status');
button42?.addEventListener('click', () => {
  const next = button42.getAttribute('aria-pressed') !== 'true';
  button42.setAttribute('aria-pressed', String(next));
  if (status42) status42.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 423 - Theme switcher

Goal: Toggle a data-theme token while respecting system preference by default.

```
<!-- HTML -->
<section class="demo theme-switcher" aria-labelledby="theme-switcher-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 43</p>
    <h2 id="theme-switcher-title">Theme switcher</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.theme-switcher {
  --accent: hsl(221 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.theme-switcher .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.theme-switcher .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .theme-switcher .demo__content { transition: transform 180ms ease; }
  .theme-switcher:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root43 = document.querySelector('.theme-switcher');
const button43 = root43?.querySelector('.demo__action');
const status43 = root43?.querySelector('.demo__status');
button43?.addEventListener('click', () => {
  const next = button43.getAttribute('aria-pressed') !== 'true';
  button43.setAttribute('aria-pressed', String(next));
  if (status43) status43.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 424 - Copy-code button

Goal: Copy a code block and announce completion in a status region.

```
<!-- HTML -->
<section class="demo copy-code-button" aria-labelledby="copy-code-button-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 44</p>
    <h2 id="copy-code-button-title">Copy-code button</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.copy-code-button {
  --accent: hsl(268 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.copy-code-button .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.copy-code-button .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .copy-code-button .demo__content { transition: transform 180ms ease; }
  .copy-code-button:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root44 = document.querySelector('.copy-code-button');
const button44 = root44?.querySelector('.demo__action');
const status44 = root44?.querySelector('.demo__status');
button44?.addEventListener('click', () => {
  const next = button44.getAttribute('aria-pressed') !== 'true';
  button44.setAttribute('aria-pressed', String(next));
  if (status44) status44.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 425 - Back-to-top control

Goal: Reveal a real button after scrolling and honor reduced motion.

```
<!-- HTML -->
<section class="demo back-to-top-control" aria-labelledby="back-to-top-control-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 45</p>
    <h2 id="back-to-top-control-title">Back-to-top control</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.back-to-top-control {
  --accent: hsl(315 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.back-to-top-control .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.back-to-top-control .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .back-to-top-control .demo__content { transition: transform 180ms ease; }
  .back-to-top-control:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root45 = document.querySelector('.back-to-top-control');
const button45 = root45?.querySelector('.demo__action');
const status45 = root45?.querySelector('.demo__status');
button45?.addEventListener('click', () => {
  const next = button45.getAttribute('aria-pressed') !== 'true';
  button45.setAttribute('aria-pressed', String(next));
  if (status45) status45.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 426 - Reading progress bar

Goal: Compute progress from document scroll while avoiding layout thrashing.

```
<!-- HTML -->
<section class="demo reading-progress-bar" aria-labelledby="reading-progress-bar-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 46</p>
    <h2 id="reading-progress-bar-title">Reading progress bar</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.reading-progress-bar {
  --accent: hsl(2 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.reading-progress-bar .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.reading-progress-bar .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .reading-progress-bar .demo__content { transition: transform 180ms ease; }
  .reading-progress-bar:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root46 = document.querySelector('.reading-progress-bar');
const button46 = root46?.querySelector('.demo__action');
const status46 = root46?.querySelector('.demo__status');
button46?.addEventListener('click', () => {
  const next = button46.getAttribute('aria-pressed') !== 'true';
  button46.setAttribute('aria-pressed', String(next));
  if (status46) status46.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 427 - Lazy media gallery

Goal: Combine loading=lazy with fixed dimensions to reduce layout shift.

```
<!-- HTML -->
<section class="demo lazy-media-gallery" aria-labelledby="lazy-media-gallery-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 47</p>
    <h2 id="lazy-media-gallery-title">Lazy media gallery</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.lazy-media-gallery {
  --accent: hsl(49 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.lazy-media-gallery .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.lazy-media-gallery .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .lazy-media-gallery .demo__content { transition: transform 180ms ease; }
  .lazy-media-gallery:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root47 = document.querySelector('.lazy-media-gallery');
const button47 = root47?.querySelector('.demo__action');
const status47 = root47?.querySelector('.demo__status');
button47?.addEventListener('click', () => {
  const next = button47.getAttribute('aria-pressed') !== 'true';
  button47.setAttribute('aria-pressed', String(next));
  if (status47) status47.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 428 - Bento grid

Goal: Use Grid spans at wide sizes and a single reading order on narrow screens.

```
<!-- HTML -->
<section class="demo bento-grid" aria-labelledby="bento-grid-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 48</p>
    <h2 id="bento-grid-title">Bento grid</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.bento-grid {
  --accent: hsl(96 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.bento-grid .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.bento-grid .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .bento-grid .demo__content { transition: transform 180ms ease; }
  .bento-grid:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root48 = document.querySelector('.bento-grid');
const button48 = root48?.querySelector('.demo__action');
const status48 = root48?.querySelector('.demo__status');
button48?.addEventListener('click', () => {
  const next = button48.getAttribute('aria-pressed') !== 'true';
  button48.setAttribute('aria-pressed', String(next));
  if (status48) status48.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 429 - Marquee alternative

Goal: Create an optional looping strip that stops for reduced-motion users.

```
<!-- HTML -->
<section class="demo marquee-alternative" aria-labelledby="marquee-alternative-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 49</p>
    <h2 id="marquee-alternative-title">Marquee alternative</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.marquee-alternative {
  --accent: hsl(143 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.marquee-alternative .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.marquee-alternative .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .marquee-alternative .demo__content { transition: transform 180ms ease; }
  .marquee-alternative:has(.demo__action[aria-pressed="true"]) .demo__content { transform:
    translateY(-4px); }
}

// JavaScript
const root49 = document.querySelector('.marquee-alternative');
const button49 = root49?.querySelector('.demo__action');
const status49 = root49?.querySelector('.demo__status');
button49?.addEventListener('click', () => {
  const next = button49.getAttribute('aria-pressed') !== 'true';
  button49.setAttribute('aria-pressed', String(next));
  if (status49) status49.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Snippet 430 - Accessible carousel controls

Goal: Use labeled buttons, a slide status region, and no forced autoplay.

```
<!-- HTML -->
<section class="demo accessible-carousel-controls" aria-labelledby="accessible-carousel-
controls-title">
  <div class="demo__content">
    <p class="demo__eyebrow">Mini-project 50</p>
    <h2 id="accessible-carousel-controls-title">Accessible carousel controls</h2>
    <p>Replace this text with the real content for the component.</p>
    <button class="demo__action" type="button" aria-pressed="false">Try interaction</button>
    <p class="demo__status" role="status" aria-live="polite"></p>
  </div>
</section>

/* CSS */
.accessible-carousel-controls {
  --accent: hsl(190 78% 56%);
  display: grid;
  place-items: center;
  min-block-size: min(70dvh, 42rem);
  padding: clamp(1rem, 5vw, 4rem);
  background: linear-gradient(135deg, color-mix(in srgb, var(--accent) 18%, #fff), #fff);
}
.accessible-carousel-controls .demo__content {
  inline-size: min(100%, 50rem);
  padding: clamp(1.25rem, 4vw, 3rem);
  border: 1px solid #0002;
  border-radius: 1.25rem;
  background: #fff;
  box-shadow: 0 18px 60px #0002;
}
.accessible-carousel-controls .demo__action { min-block-size: 44px; padding-inline: 1rem; }
@media (prefers-reduced-motion: no-preference) {
  .accessible-carousel-controls .demo__content { transition: transform 180ms ease; }
  .accessible-carousel-controls:has(.demo__action[aria-pressed="true"]) .demo__content {
    transform: translateY(-4px); }
}

// JavaScript
const root50 = document.querySelector('.accessible-carousel-controls');
const button50 = root50?.querySelector('.demo__action');
const status50 = root50?.querySelector('.demo__status');
button50?.addEventListener('click', () => {
  const next = button50.getAttribute('aria-pressed') !== 'true';
  button50.setAttribute('aria-pressed', String(next));
  if (status50) status50.textContent = next ? 'Demo state enabled.' : 'Demo state disabled.';
});
```

Step by step

1. Create a blank document and paste the HTML portion into body, the CSS portion into a stylesheet, and the JavaScript portion into a module script.
2. Keep the semantic title, real button, status region, focus behavior, and 44px minimum control target while changing the visual design.
3. Replace the generic content with the content/controls named in the project title; use native HTML elements before inventing custom roles.
4. Test the component at 320px and 1440px, at 200% zoom, with keyboard only, and with reduced-motion enabled.
5. Only after the interaction works without animation should you add richer transforms, images, or library effects.

Verify: Click the action, press it with Enter and Space, and inspect aria-pressed/status changes. Resize and enable prefers-reduced-motion to verify the layout and motion fallback.

Gotcha: These mini-projects are original teaching scaffolds inspired by common interface ideas (including the screenshots), not copied source from the creators. For production, specialize the structure and behavior to the component's actual semantics.

Part III - Build discipline checklist

1. Start with valid semantic HTML and a keyboard-usable interaction.
2. Add CSS from the small-screen/default state outward; use Grid/Flexbox for layout and transforms/opacity for motion.
3. Add JavaScript only for state/behavior that HTML and CSS cannot express cleanly.
4. Treat user input, URLs, storage, network data, and copied third-party snippets as untrusted until validated.
5. Test at 200% zoom, narrow width, keyboard-only, reduced motion, dark/light schemes, slow network, and offline/failure paths when relevant.
6. Record source URL, exact version/commit, license, notices, and dependencies for every third-party inclusion.
7. Re-test after customization; a copied component can look correct while losing accessibility or state behavior.

End of handbook